

CHAPTER 6

Type Checking

A compiler must check that the source program follows both the syntactic and semantic conventions of the source language. This checking, called *static checking* (to distinguish it from *dynamic* checking during execution of the target program), ensures that certain kinds of programming errors will be detected and reported. Examples of static checks include:

1. *Type checks.* A compiler should report an error if an operator is applied to an incompatible operand; for example, if an array variable and a function variable are added together.
2. *Flow-of-control checks.* Statements that cause flow of control to leave a construct must have some place to which to transfer the flow of control. For example, a break statement in C causes control to leave the smallest enclosing while, for, or switch statement; an error occurs if such an enclosing statement does not exist.
3. *Uniqueness checks.* There are situations in which an object must be defined exactly once. For example, in Pascal, an identifier must be declared uniquely, labels in a case statement must be distinct, and elements in a scalar type may not be repeated.
4. *Name-related checks.* Sometimes, the same name must appear two or more times. For example, in Ada, a loop or block may have a name that appears at the beginning and end of the construct. The compiler must check that the same name is used at both places.

In this chapter, we focus on type checking. As the above examples indicate, most of the other static checks are routine and can be implemented using the techniques of the last chapter. Some of them can be folded into other activities. For example, as we enter information about a name into a symbol table, we can check that the name is declared uniquely. Many Pascal compilers combine static checking and intermediate code generation with parsing. With more complex constructs, like those of Ada, it may be convenient to have a separate type-checking pass between parsing and intermediate code generation, as indicated in Fig. 6.1.

A type checker verifies that the type of a construct matches that expected by

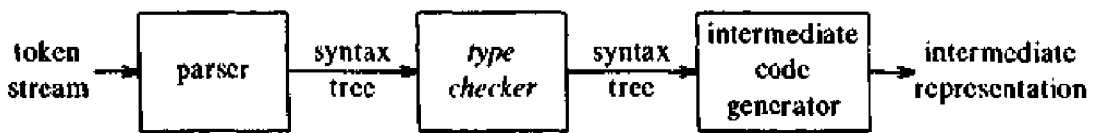


Fig. 6.1. Position of type checker.

its context. For example, the built-in arithmetic operator `mod` in Pascal requires integer operands, so a type checker must verify that the operands of `mod` have type integer. Similarly, the type checker must verify that dereferencing is applied only to a pointer, that indexing is done only on an array, that a user-defined function is applied to the correct number and type of arguments, and so forth. A specification of a simple type checker appears in Section 6.2. The representation of types and the question of when two types match are discussed in Section 6.3.

Type information gathered by a type checker may be needed when code is generated. For example, arithmetic operators like `+` usually apply to either integers or reals, perhaps to other types, and we have to look at the context of `+` to determine the sense that is intended. A symbol that can represent different operations in different contexts is said to be "overloaded." Overloading may be accompanied by coercion of types, where a compiler supplies an operator to convert an operand into the type expected by the context.

A distinct notion from overloading is that of "polymorphism." The body of a polymorphic function can be executed with arguments of several types. A unification algorithm for inferring types of polymorphic functions concludes this chapter.

6.1 TYPE SYSTEMS

The design of a type checker for a language is based on information about the syntactic constructs in the language, the notion of types, and the rules for assigning types to language constructs. The following excerpts from the Pascal report and the C reference manual, respectively, are examples of information that a compiler writer might have to start with.

- "If both operands of the arithmetic operators of addition, subtraction and multiplication are of type integer, then the result is of type integer."
- "The result of the unary `&` operator is a pointer to the object referred to by the operand. If the type of the operand is `'...'`, the type of the result is `'pointer to ...'`."

Implicit in the above excerpts is the idea that each expression has a type

associated with it. Furthermore, types have structure; the type "pointer to ..." is constructed from the type that "..." refers to.

In both Pascal and C, types are either basic or constructed. Basic types are the atomic types with no internal structure as far as the programmer is concerned. In Pascal, the basic types are boolean, character, integer, and real. Subrange types, like 1..10, and enumerated types, like

(violet, indigo, blue, green, yellow, orange, red)

can be treated as basic types. Pascal allows a programmer to construct types from basic types and other constructed types, with arrays, records, and sets being examples. In addition, pointers and functions can also be treated as constructed types.

Type Expressions

The type of a language construct will be denoted by a "type expression." Informally, a type expression is either a basic type or is formed by applying an operator called a *type constructor* to other type expressions. The sets of basic types and constructors depend on the language to be checked.

This chapter uses the following definition of *type expressions*:

1. A basic type is a type expression. Among the basic types are *boolean*, *char*, *integer*, and *real*. A special basic type, *type_error*, will signal an error during type checking. Finally, a basic type *void* denoting "the absence of a value" allows statements to be checked.
2. Since type expressions may be named, a type name is a type expression. An example of the use of type names appears in 3(c) below; type expressions containing names are discussed in Section 6.3.
3. A type constructor applied to type expressions is a type expression. Constructors include:

- a) *Arrays*. If T is a type expression, then $\text{array}(I, T)$ is a type expression denoting the type of an array with elements of type T and index set I . I is often a range of integers. For example, the Pascal declaration

var A: array[1..10] of integer;

associates the type expression $\text{array}(1..10, \text{integer})$ with A.

- b) *Products*. If T_1 and T_2 are type expressions, then their Cartesian product $T_1 \times T_2$ is a type expression. We assume that $\times$ associates to the left.
- c) *Records*. The difference between a record and a product is that the fields of a record have names. The *record* type constructor will be applied to a tuple formed from field names and field types. (Technically, the field names should be part of the type constructor, but it is

convenient to keep field names together with their associated types. In Chapter 8, the type constructor *record* is applied to a pointer to a symbol table containing entries for the field names.) For example, the Pascal program fragment

```
type row = record
    address: integer;
    lexeme: array [1..15] of char
end;
var table: array [1..101] of row;
```

declares the type name *row* representing the type expression

$$\text{record}((\text{address} \times \text{integer}) \times (\text{lexeme} \times \text{array}(1..15, \text{char})))$$

and the variable *table* to be an array of records of this type.

- d) *Pointers*. If T is a type expression, then $\text{pointer}(T)$ is a type expression denoting the type "pointer to an object of type T ." For example, in Pascal, the declaration

```
var p: ↑ row
```

declares variable *p* to have type $\text{pointer}(\text{row})$.

- e) *Functions*. Mathematically, a function maps elements of one set, the *domain*, to another set, the *range*. We may treat functions in programming languages as mapping a *domain type* D to a *range type* R . The type of such a function will be denoted by the type expression $D \rightarrow R$. For example, the built-in function *mod* of Pascal has domain type $\text{int} \times \text{int}$, i.e., a pair of integers, and range type int . Thus, we say *mod* has the type¹

$$\text{int} \times \text{int} \rightarrow \text{int}$$

As another example, the Pascal declaration

```
function f(a, b: char) : ↑ integer; ...
```

says that the domain type of *f* is denoted by $\text{char} \times \text{char}$ and range type by $\text{pointer}(\text{integer})$. The type of *f* is thus denoted by the type expression

$$\text{char} \times \text{char} \rightarrow \text{pointer}(\text{integer})$$

Often, for implementation reasons discussed in the next chapter, there are limitations on the type that a function may return; e.g., no arrays or functions may be returned. However, there are languages, of which Lisp is the most prominent example, that allow functions to

¹ We assume that $\times$ has higher precedence than $\rightarrow$, so $\text{int} \times \text{int} \rightarrow \text{int}$ is the same as $(\text{int} \times \text{int}) \rightarrow \text{int}$. Also, $\rightarrow$ associates to the right.

return objects of arbitrary types, so, for example, we can define a function g of type

$$(integer \rightarrow integer) \rightarrow (integer \rightarrow integer)$$

That is, g takes as argument a function that maps an integer to an integer and g produces as a result another function of the same type.

4. Type expressions may contain variables whose values are type expressions. Type variables will be introduced in Section 6.6.

A convenient way to represent a type expressions is to use a graph. Using the syntax-directed approach of Section 5.2, we can construct a tree or a dag for a type expression, with interior nodes for type constructors and leaves for basic types, type names, and type variables (see Fig. 6.2). Examples of representations of type expressions that have been used in compilers are given in Section 6.3.



Fig. 6.2. Tree and dag, respectively, for $char \times char \rightarrow pointer(integer)$.

Type Systems

A *type system* is a collection of rules for assigning type expressions to the various parts of a program. A type checker implements a type system. The type systems in this chapter are specified in a syntax-directed manner, so they can be readily implemented using the techniques of the previous chapter.

Different type systems may be used by different compilers or processors of the same language. For example, in Pascal, the type of an array includes the index set of the array, so a function with an array argument can only be applied to arrays with that index set. Many Pascal compilers, however, allow the index set to be left unspecified when an array is passed as an argument. Thus these compilers use a different type system than that in the Pascal language definition. Similarly, in the UNIX system, the `lint` command examines C programs for possible bugs using a more detailed type system than the C compiler itself uses.

Static and Dynamic Checking of Types

Checking done by a compiler is said to be static, while checking done when the target program runs is termed dynamic. In principle, any check can be done dynamically, if the target code carries the type of an element along with

the value of that element.

A *sound* type system eliminates the need for dynamic checking for type errors because it allows us to determine statically that these errors cannot occur when the target program runs. That is, if a sound type system assigns a type other than *type_error* to a program part, then type errors cannot occur when the target code for the program part is run. A language is *strongly typed* if its compiler can guarantee that the programs it accepts will execute without type errors.

In practice, some checks can be done only dynamically. For example, if we first declare

```
table: array[0..255] of char;  
i: integer
```

and then compute `table[i]`, a compiler cannot in general guarantee that during execution, the value of `i` will lie in the range 0 to 255.²

Error Recovery

Since type checking has the potential for catching errors in programs, it is important for a type checker to do something reasonable when an error is discovered. At the very least, the compiler must report the nature and location of the error. It is desirable for the type checker to recover from errors, so it can check the rest of the input. Since error handling affects the type-checking rules, it has to be designed into the type system right from the start; the rules must be prepared to cope with errors.

The inclusion of error handling may result in a type system that goes beyond the one needed to specify correct programs. For example, once an error has occurred, we may not know the type of the incorrectly formed program fragment. Coping with missing information requires techniques similar to those needed for languages that do not require identifiers to be declared before they are used. Type variables, discussed in Section 6.6, can be used to ensure consistent usage of undeclared or apparently misdeclared identifiers.

6.2 SPECIFICATION OF A SIMPLE TYPE CHECKER

In this section, we specify a type checker for a simple language in which the type of each identifier must be declared before the identifier is used. The type checker is a translation scheme that synthesizes the type of each expression from the types of its subexpressions. The type checker can handle arrays, pointers, statements, and functions.

² Data-flow analysis techniques similar to those in Chapter 10 can be used to infer if `i` is within bounds in some programs. However, no technique can make the decision correctly in all cases.

A Simple Language

The grammar in Fig. 6.3 generates programs, represented by the nonterminal P , consisting of a sequence of declarations D followed by a single expression E .

$$\begin{aligned} P &\rightarrow D ; E \\ D &\rightarrow D ; D \mid \text{id} : T \\ T &\rightarrow \text{char} \mid \text{integer} \mid \text{array} \{ \text{num} \} \text{ of } T \mid \uparrow T \\ E &\rightarrow \text{literal} \mid \text{num} \mid \text{id} \mid E \bmod E \mid E [E] \mid E \uparrow \end{aligned}$$

Fig. 6.3. Grammar for source language.

One program generated by the grammar in Fig. 6.3 is:

```
key: integer;
key mod 1999
```

Before discussing expressions, consider the types in the language. The language itself has two basic types, *char* and *integer*; a third basic type *type_error* is used to signal errors. For simplicity, we assume that all arrays start at 1. For example,

```
array [256] of char
```

leads to the type expression *array*(1..256, *char*) consisting of the constructor *array* applied to the subrange 1..256 and the type *char*. As in Pascal, the prefix operator $\uparrow$ in declarations builds a pointer type, so

```
 $\uparrow$  integer
```

leads to the type expression *pointer*(*integer*), consisting of the constructor *pointer* applied to the type *integer*.

In the translation scheme of Fig. 6.4, the action associated with the production $D \rightarrow \text{id} : T$ saves a type in a symbol-table entry for an identifier. The action *addtype*(*id.entry*, *T.type*) is applied to synthesized attribute *entry* pointing to the symbol-table entry for *id* and a type expression represented by synthesized attribute *type* of nonterminal *T*.

If *T* generates **char** or **integer**, then *T.type* is defined to be *char* or *integer*, respectively. The upper bound of an array is obtained from the attribute *val* of token **num** that gives the integer represented by **num**. Arrays are assumed to start at 1, so the type constructor *array* is applied to the subrange 1..*num.val* and the element type.

Since *D* appears before *E* on the right side of $P \rightarrow D ; E$, we can be sure that the types of all declared identifiers will be saved before the expression generated by *E* is checked. (See Chapter 5.) In fact, by suitably modifying the grammar in Fig. 6.3, we can implement the translation schemes in this section during either top-down or bottom-up parsing, if desired.

$$\begin{array}{ll}
P \rightarrow D ; E & \\
D \rightarrow D ; D & \\
D \rightarrow \text{id} : T & \{ \text{addtype}(\text{id.entry}, T.\text{type}) \} \\
T \rightarrow \text{char} & \{ T.\text{type} := \text{char} \} \\
T \rightarrow \text{integer} & \{ T.\text{type} := \text{integer} \} \\
T \rightarrow \uparrow T_1 & \{ T.\text{type} := \text{pointer}(T_1.\text{type}) \} \\
T \rightarrow \text{array} [\text{num}] \text{ of } T_1 & \{ T.\text{type} := \text{array}(1.. \text{num.val}, T_1.\text{type}) \}
\end{array}$$

Fig. 6.4. The part of a translation scheme that saves the type of an identifier.

Type Checking of Expressions

In the following rules, the synthesized attribute *type* for *E* gives the type expression assigned by the type system to the expression generated by *E*. The following semantic rules say that constants represented by the tokens **literal** and **num** have type *char* and *integer*, respectively:

$$\begin{array}{ll}
E \rightarrow \text{literal} & \{ E.\text{type} := \text{char} \} \\
E \rightarrow \text{num} & \{ E.\text{type} := \text{integer} \}
\end{array}$$

We use a function *lookup*(*e*) to fetch the type saved in the symbol-table entry pointed to by *e*. When an identifier appears in an expression, its declared type is fetched and assigned to the attribute *type*:

$$E \rightarrow \text{id} \quad \{ E.\text{type} := \text{lookup}(\text{id.entry}) \}$$

The expression formed by applying the mod operator to two subexpressions of type *integer* has type *integer*; otherwise, its type is *type_error*. The rule is

$$\begin{array}{l}
E \rightarrow E_1 \text{ mod } E_2 \quad \{ E.\text{type} := \text{if } E_1.\text{type} = \text{integer} \text{ and} \\
\hspace{15em} E_2.\text{type} = \text{integer} \text{ then } \text{integer} \\
\hspace{15em} \text{else } \text{type_error} \}
\end{array}$$

In an array reference $E_1 [E_2]$, the index expression E_2 must have type *integer*, in which case the result is the element type *t* obtained from the type *array*(*s*, *t*) of E_1 ; we make no use of the index set *s* of the array.

$$\begin{array}{l}
E \rightarrow E_1 [E_2] \quad \{ E.\text{type} := \text{if } E_2.\text{type} = \text{integer} \text{ and} \\
\hspace{15em} E_1.\text{type} = \text{array}(s, t) \text{ then } t \\
\hspace{15em} \text{else } \text{type_error} \}
\end{array}$$

Within expressions, the postfix operator $\uparrow$ yields the object pointed to by its operand. The type of $E \uparrow$ is the type *t* of the object pointed to by the pointer *E*:

$$\begin{array}{l}
E \rightarrow E_1 \uparrow \quad \{ E.\text{type} := \text{if } E_1.\text{type} = \text{pointer}(t) \text{ then } t \\
\hspace{15em} \text{else } \text{type_error} \}
\end{array}$$

We leave it to the reader to add productions and semantic rules to permit

additional types and operations within expressions. For example, to allow identifiers to have the type *boolean*, we can introduce the production $T \rightarrow \text{boolean}$ to the grammar of Fig. 6.3. The introduction of comparison operators like $<$ and logical connectives like **and** into the productions for E would allow the construction of expressions of type *boolean*.

Type Checking of Statements

Since language constructs like statements typically do not have values, the special basic type *void* can be assigned to them. If an error is detected within a statement, the type assigned to the statement is *type_error*.

The statements we consider are assignment, conditional, and while statements. Sequences of statements are separated by semicolons. The productions in Fig. 6.5 can be combined with those of Fig. 6.3 if we change the production for a complete program to $P \rightarrow D ; S$. A program now consists of declarations followed by statements; the above rules for checking expressions are still needed because statements can have expressions within them.

$S \rightarrow \text{id} := E$	{ $S.type :=$ if $\text{id}.type = E.type$ then <i>void</i> else <i>type_error</i> }
$S \rightarrow \text{if } E \text{ then } S_1$	{ $S.type :=$ if $E.type = \text{boolean}$ then $S_1.type$ else <i>type_error</i> }
$S \rightarrow \text{while } E \text{ do } S_1$	{ $S.type :=$ if $E.type = \text{boolean}$ then $S_1.type$ else <i>type_error</i> }
$S \rightarrow S_1 ; S_2$	{ $S.type :=$ if $S_1.type = \text{void}$ and $S_2.type = \text{void}$ then <i>void</i> else <i>type_error</i> }

Fig. 6.5. Translation scheme for checking the type of statements.

Rules for checking statements are given in Fig. 6.5. The first rule checks that the left and right sides of an assignment statement have the same type.³ The second and third rules specify that expressions in conditional and while statements must have type *boolean*. Errors are propagated by the last rule in Fig. 6.5 because a sequence of statements has type *void* only if each sub-statement has type *void*. In these rules, a mismatch of types produces the type *type_error*; a friendly type checker would, of course, report the nature and location of the type mismatch as well.

³ If an expression is allowed on the left side of an assignment, then we also have to distinguish between *l*-values and *r*-values. For example, $1 := 2$ is incorrect because the constant 1 cannot be assigned to.

Type Checking of Functions

The application of a function to an argument can be captured by the production

$$E \rightarrow E (E)$$

in which an expression is the application of one expression to another. The rules for associating type expressions with nonterminal T can be augmented by the following production and action to permit function types in declarations.

$$T \rightarrow T_1 \rightarrow T_2 \quad \{ T.type := T_1.type \rightarrow T_2.type \}$$

Quotes around the arrow used as a function constructor distinguish it from the arrow used as the metasymbol in a production.

The rule for checking the type of a function application is

$$E \rightarrow E_1 (E_2) \quad \{ E.type := \text{if } E_2.type = s \text{ and } E_1.type = s \rightarrow t \text{ then } t \\ \text{else type_error} \}$$

This rule says that in an expression formed by applying E_1 to E_2 , the type of E_1 must be a function $s \rightarrow t$ from the type s of E_2 to some range type t ; the type of $E_1 (E_2)$ is t .

Many issues related to type checking in the presence of functions can be discussed with respect to the simple syntax above. The generalization to functions with more than one argument is done by constructing a product type consisting of the arguments. Note that n arguments of type $T_1, \dots, T_n$ can be viewed as a single argument of type $T_1 \times \dots \times T_n$. For example, we might write

$$\text{root} : (\text{real} \rightarrow \text{real}) \times \text{real} \rightarrow \text{real} \quad (6.1)$$

to declare a function `root` that takes a function from reals to reals and a real as arguments and returns a real. Pascal-like syntax for this declaration is

```
function root (function f (real): real; x: real): real
```

The syntax in (6.1) separates the declaration of the type of a function from the names of its parameters.

6.3 EQUIVALENCE OF TYPE EXPRESSIONS

The checking rules in the last section have the form, "if two type expressions are equal then return a certain type else return `type_error`." It is therefore important to have a precise definition of when two type expressions are equivalent. Potential ambiguities arise when names are given to type expressions and the names are then used in subsequent type expressions. The key issue is whether a name in a type expression stands for itself or whether it is an abbreviation for another type expression.

Since there is interaction between the notion of equivalence of types and the

representation of types, we shall talk about both together. For efficiency, compilers use representations that allow type equivalence to be determined quickly. The notion of type equivalence implemented by a specific compiler can often be explained using the concepts of structural and name equivalence discussed in this section. The discussion is in terms of a graph representation of type expressions with leaves for basic types and type names, and interior nodes for type constructors, as in Fig. 6.2. As we shall see, recursively defined types lead to cycles in the type graph if a name is treated as an abbreviation for a type expression.

Structural Equivalence of Type Expressions

As long as type expressions are built from basic types and constructors, a natural notion of equivalence between two type expressions is *structural equivalence*; i.e., two expressions are either the same basic type, or are formed by applying the same constructor to structurally equivalent types. That is, two type expressions are structurally equivalent if and only if they are identical. For example, the type expression *integer* is equivalent only to *integer* because they are the same basic type. Similarly, *pointer(integer)* is equivalent only to *pointer(integer)* because the two are formed by applying the same constructor *pointer* to equivalent types. If we use the value-number method of Algorithm 5.1 to construct a dag representation of type expressions, then identical type expressions will be represented by the same node.

Modifications of the notion of structural equivalence are often needed in practice to reflect the actual type-checking rules of the source language. For example, when arrays are passed as parameters, we may not wish to include the array bounds as part of the type.

The algorithm for testing structural equivalence in Fig. 6.6 can be adapted to test modified notions of equivalence. It assumes that the only type constructors are for arrays, products, pointers, and functions. The algorithm recursively compares the structure of type expressions without checking for cycles so it can be applied to a tree or a dag representation. Identical type expressions do not need to be represented by the same node in the dag. Structural equivalence of nodes in type graphs with cycles can be tested using an algorithm in Section 6.7.

The array bounds s_1 and t_1 in

$$\begin{aligned}s &= \text{array}(s_1, s_2) \\ t &= \text{array}(t_1, t_2)\end{aligned}$$

are ignored if the test for array equivalence in lines 4 and 5 of Fig. 6.6 is reformulated as

else if $s = \text{array}(s_1, s_2)$ **and** $t = \text{array}(t_1, t_2)$ **then**
 return *sequiv*(s_2, t_2)

In certain situations, we can find a representation for type expressions that is significantly more compact than the type graph notation. In the next

```

(1) function sequiv(s, t): boolean;
    begin
(2)     if s and t are the same basic type then
(3)         return true
(4)     else if s = array(s1, s2) and t = array(t1, t2) then
(5)         return sequiv(s1, t1) and sequiv(s2, t2)
(6)     else if s = s1 × s2 and t = t1 × t2 then
(7)         return sequiv(s1, t1) and sequiv(s2, t2)
(8)     else if s = pointer(s1) and t = pointer(t1) then
(9)         return sequiv(s1, t1)
(10)    else if s = s1 → s2 and t = t1 → t2 then
(11)        return sequiv(s1, t1) and sequiv(s2, t2)
    else
(12)        return false
    end

```

Fig. 6.6. Testing the structural equivalence of two type expressions *s* and *t*.

example, some of the information from a type expression is encoded as a sequence of bits, which can then be interpreted as a single integer. The encoding is such that distinct integers represent structurally inequivalent type expressions. The test for structural equivalence can be accelerated by first testing for structural inequivalence by comparing the integer representations of the types, and then applying the algorithm of Fig. 6.6 only if the integers are the same.

Example 6.1. The encoding of type expressions in this example is from a C compiler written by D. M. Ritchie. It is also used by the C compiler described in Johnson [1979].

Consider type expressions with the following type constructors for pointers, functions, and arrays: *pointer*(*t*) denotes a pointer to type *t*, *freturns*(*t*) denotes a function of some arguments that returns an object of type *t*, and *array*(*t*) denotes an array (of some indeterminate length) of elements of type *t*. Notice that we have simplified the array and function type constructors. We shall keep track of the number of elements in an array, but the number will be kept elsewhere, so it is not part of the type constructor *array*. Similarly, the only operand of the constructor *freturns* is the type of the result of a function; the types of the function arguments will be stored elsewhere. Thus, objects with structurally equivalent expressions of this type system might still fail to meet the test of Fig. 6.6 applied to the more detailed type system used there.

Since each of these constructors is a unary operator, type expressions formed by applying these constructors to basic types have a very uniform structure. Examples of such type expressions are:

char
freturns(char)
pointer(freturns(char))
array(pointer(freturns(char)))

Each of the above expressions can be represented by a sequence of bits using a simple encoding scheme. Since there are only three type constructors, we can use two bits to encode a constructor, as follows:

TYPE CONSTRUCTOR	ENCODING
<i>pointer</i>	01
<i>array</i>	10
<i>freturns</i>	11

The basic types of C are encoded using four bits in Johnson [1979]; our four basic types might be encoded as:

BASIC TYPE	ENCODING
<i>boolean</i>	0000
<i>char</i>	0001
<i>integer</i>	0010
<i>real</i>	0011

Restricted type expressions can now be encoded as sequences of bits. The rightmost four bits encode the basic type in a type expression. Moving from right to left, the next two bits indicate the constructor applied to the basic type, the next two bits describe the constructor applied to that, and so on. For example,

TYPE EXPRESSION	ENCODING
<i>char</i>	000000 0001
<i>freturns(char)</i>	000011 0001
<i>pointer(freturns(char))</i>	000111 0001
<i>array(pointer(freturns(char)))</i>	100111 0001

See Exercise 6.12 for more details.

Besides saving space, such a representation keeps track of the constructors that appear in any type expression. Two different bit sequences cannot represent the same type because either the basic type or the constructors in the type expressions are different. Of course, different types could have the same bit sequence since array size and function arguments are not represented.

The encoding in this example can be extended to include record types. The idea is to treat each record as a basic type in the encoding; a separate sequence of bits encodes the type of each field of the record. Type equivalence in C is examined further in Example 6.4. □

Names for Type Expressions

In some languages, types can be given names. For example, in the Pascal program fragment

```

type link = ↑ cell;
var  next : link;
     last : link;
     p    : ↑ cell;
     q, r : ↑ cell;

```

(6.2)

the identifier `link` is declared to be a name for the type `↑ cell`. The question arises, do the variables `next`, `last`, `p`, `q`, `r` all have identical types? Surprisingly, the answer depends on the implementation. The problem arose because the Pascal Report did not define the term “identical type.”

To model this situation, we allow type expressions to be named and allow these names to appear in type expressions where we previously had only basic types. For example, if `cell` is the name of a type expression, then `pointer(cell)` is a type expression. For the time being, suppose there are no circular type expression definitions such as defining `cell` to be the name of a type expression containing `cell`.

When names are allowed in type expressions, two notions of equivalence of type expressions arise, depending on the treatment of names. *Name equivalence* views each type name as a distinct type, so two type expressions are name equivalent if and only if they are identical. Under *structural equivalence*, names are replaced by the type expressions they define, so two type expressions are structurally equivalent if they represent two structurally equivalent type expressions when all names have been substituted out.

Example 6.2. The type expressions that might be associated with the variables in the declarations (6.2) are given in the following table.

VARIABLE	TYPE EXPRESSION
<code>next</code>	<code>link</code>
<code>last</code>	<code>link</code>
<code>p</code>	<code>pointer(cell)</code>
<code>q</code>	<code>pointer(cell)</code>
<code>r</code>	<code>pointer(cell)</code>

Under name equivalence, the variables `next` and `last` have the same type because they have the same associated type expressions. The variables `p`, `q`, and `r` also have the same type, but `p` and `next` do not, since their associated type expressions are different. Under structural equivalence, all five variables have the same type because `link` is a name for the type expression `pointer(cell)`. □

The concepts of structural and name equivalence are useful in explaining the rules used by various languages to associate types with identifiers through declarations.

Example 6.3. Confusion arises in Pascal from the fact that many implementations associate an implicit type name with each declared identifier. If the declaration contains a type expression that is not a name, an implicit name is created. A fresh implicit name is created every time a type expression appears in a variable declaration.

Thus, implicit names are created for the type expressions in the two declarations containing *p*, *q*, and *r*, in (6.2). That is, the declarations are treated as if they were

```

type link = ↑ cell;
  np    = ↑ cell;
  nqr   = ↑ cell;
var next : link;
  last  : link;
  p     : np;
  q     : nqr;
  r     : nqr;

```

Here, new type names *np* and *nqr* have been introduced. Under name equivalence, since *next* and *last* are declared with the same type name, they are treated as having equivalent types. Similarly, *q* and *r* are treated as having equivalent types because the same implicit type name is associated with them. However, *p*, *q*, and *next* do not have equivalent types, since they all have types with different names.

The typical implementation is to construct a type graph to represent the types. Every time a type constructor or basic type is seen, a new node is created. Every time a new type name is seen, a leaf is created, however, we keep track of the type expression to which the name refers. With this representation, two type expressions are equivalent if they are represented by the same node in the type graph. Figure 6.7 shows a type graph for the declarations (6.2). Dotted lines show the association between variables and nodes in the type graph. Note that type name *cell* has three parents, all labeled *pointer*. An equal sign appears between the type name *link* and the node in the type graph to which it refers. □

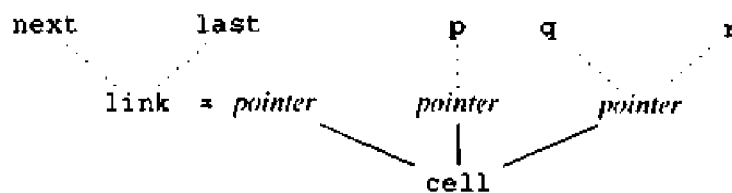


Fig. 6.7. Association of variables and nodes in the type graph.

Cycles in Representations of Types

Basic data structures like linked lists and trees are often defined recursively; e.g., a linked list is either empty or consists of a cell with a pointer to a linked list. Such data structures are usually implemented using records that contain pointers to similar records, and type names play an essential role in defining the types of such records.

Consider a linked list of cells, each containing some integer information and a pointer to the next cell in the list. Pascal declarations of type names corresponding to links and cells are:

```

type link = ↑ cell;
   cell = record
       info : integer;
       next : link
   end;

```

Note that the type name `link` is defined in terms of `cell` and that `cell` is defined in terms of `link`, so their definitions are recursive.

Recursively defined type names can be substituted out if we are willing to introduce cycles into the type graph. If `pointer(cell)` is substituted for `link`, the type expression shown in Fig. 6.8(a) is obtained for `cell`. Using cycles as in Fig. 6.8(b), we can eliminate mention of `cell` from the part of the type graph below the node labeled `record`.

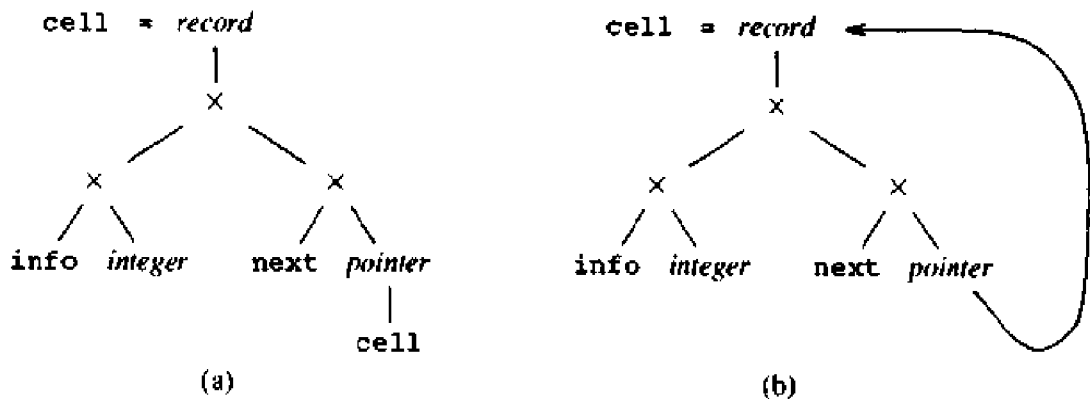


Fig. 6.8. Recursively defined type name `cell`.

Example 6.4. C avoids cycles in type graphs by using structural equivalence for all types except records. In C, the declaration of `cell` would look like

```

struct cell {
    int info;
    struct cell *next;
};

```

C uses the keyword `struct` rather than `record`, and the name `cell` becomes part of the type of the record. In effect, C uses the acyclic

representation in Fig. 6.8(a).

C requires type names to be declared before they are used, with the exception of allowing pointers to undeclared record types. All potential cycles, therefore, are due to pointers to records. Since the name of a record is part of its type, testing for structural equivalence stops when a record constructor is reached — either the types being compared are equivalent because they are the same named record type or they are inequivalent. $\square$

6.4 TYPE CONVERSIONS

Consider expressions like $x + i$ where x is of type real and i is of type integer. Since the representation of integers and reals is different within a computer, and different machine instructions are used for operations on integers and reals, the compiler may have to first convert one of the operands of $+$ to ensure that both operands are of the same type when the addition takes place.

The language definition specifies what conversions are necessary. When an integer is assigned to a real, or vice versa, the conversion is to the type of the left side of the assignment. In expressions, the usual transformation is to convert the integer into a real number and then perform a real operation on the resulting pair of real operands. The type checker in a compiler can be used to insert these conversion operations into the intermediate representation of the source program. For example, postfix notation for $x + i$, might be

$x \ i \ \text{inttoreal} \ \text{real} +$

Here, the **inttoreal** operator converts i from integer to real and then **real+** performs real addition on its operands.

Type conversion often arises in another context. A symbol having different meanings depending on its context is said to be overloaded. Overloading will be discussed in the next section, but it is mentioned because type conversions often accompany overloading.

Coercions

Conversion from one type to another is said to be *implicit* if it is to be done automatically by the compiler. Implicit type conversions, also called *coercions*, are limited in many languages to situations where no information is lost in principle; e.g., an integer may be converted to a real but not vice-versa. In practice, however, loss is possible when a real number must fit into the same number of bits as an integer.

Conversion is said to be *explicit* if the programmer must write something to cause the conversion. For all practical purposes, all conversions in Ada are explicit. Explicit conversions look just like function applications to a type checker, so they present no new problems.

For example, in Pascal, a built-in function **ord** maps a character to an integer, and **chr** does the inverse mapping from an integer to a character, so

these conversions are explicit. C, on the other hand, coerces (i.e., implicitly converts) ASCII characters to integers between 0 and 127 in arithmetic expressions.

Example 6.5. Consider expressions formed by applying an arithmetic operator **op** to constants and identifiers, as in the grammar of Fig. 6.9. Suppose there are two types — real and integer, with integers converted to reals when necessary. Attribute *type* of nonterminal *E* can be either integer or real, and the type-checking rules are shown in Fig. 6.9. As in Section 6.2, function *lookup(e)* returns the type saved in the symbol-table entry pointed to by *e*. □

PRODUCTION	SEMANTIC RULE
$E \rightarrow \text{num}$	$E.type := \text{integer}$
$E \rightarrow \text{num} . \text{num}$	$E.type := \text{real}$
$E \rightarrow \text{id}$	$E.type := \text{lookup}(\text{id.entry})$
$E \rightarrow E_1 \text{ op } E_2$	$E.type :=$ if $E_1.type = \text{integer}$ and $E_2.type = \text{integer}$ then <i>integer</i> else if $E_1.type = \text{integer}$ and $E_2.type = \text{real}$ then <i>real</i> else if $E_1.type = \text{real}$ and $E_2.type = \text{integer}$ then <i>real</i> else if $E_1.type = \text{real}$ and $E_2.type = \text{real}$ then <i>real</i> else <i>type_error</i>

Fig. 6.9. Type-checking rules for coercion from integer to real.

Implicit conversion of constants can usually be done at compile time, often with a great improvement to the running time of the object program. In the following code fragments, **X** is an array of reals that is being initialized to all 1's. Using one Pascal compiler, Bentley [1982] found that the code fragment

```
for I := 1 to N do X[I] := 1
```

took 48.4N microseconds, while the fragment

```
for I := 1 to N do X[I] := 1.0
```

took 5.4N microseconds. Both fragments assign the value one to elements of an array of reals. However, the code generated (by this compiler) for the first fragment contained a call to a run-time routine to convert the integer representation of 1 into the real-number representation. Since it is known at compile time that **X** is an array of reals, a more thorough compiler would convert 1 to 1.0 at compile time.

6.5 OVERLOADING OF FUNCTIONS AND OPERATORS

An *overloaded* symbol is one that has different meanings depending on its context. In mathematics, the addition operator $+$ is overloaded, because $+$ in $A + B$ has different meanings when A and B are integers, reals, complex numbers, or matrices. In Ada, parentheses $()$ are overloaded; the expression $A(I)$ can be the I th element of the array A , a call to function A with argument I , or an explicit conversion of expression I to type A .

Overloading is *resolved* when a unique meaning for an occurrence of an overloaded symbol is determined. For example, if $+$ can denote either integer addition or real addition, then the two occurrences of $+$ in $x + (i + j)$ can denote different forms of addition, depending on the types of x , i , and j . The resolution of overloading is sometimes referred to as *operator identification*, because it determines which operation an operator symbol denotes.

The arithmetic operators are overloaded in most languages. However, overloading involving arithmetic operators like $+$ can be resolved by looking only at the arguments of the operator. The case analysis for determining whether to use the integer or real version of $+$ is similar to that in the semantic rule for $E \rightarrow E_1 \text{ op } E_2$ in Fig. 6.9, where the type of E is determined by looking at the possible types of E_1 and E_2 .

Set of Possible Types for a Subexpression

It is not always possible to resolve overloading by looking only at the arguments of a function, as the next example shows. Instead of a single type, a subexpression standing alone may have a set of possible types. In Ada, the context must provide sufficient information to narrow the choice down to a single type.

Example 6.6. In Ada, one of the standard (i.e., built-in) interpretations of the operator $*$ is that of a function from a pair of integers to an integer. The operator can be overloaded by adding declarations like the following:

```
function "*" ( i, j : integer ) return complex;
function "*" ( x, y : complex ) return complex;
```

After the above declarations, the possible types for $*$ include:

```
integer  $\times$  integer  $\rightarrow$  integer
integer  $\times$  integer  $\rightarrow$  complex
complex  $\times$  complex  $\rightarrow$  complex
```

Suppose that the only possible type for 2, 3, and 5 is integer. With the above declarations, the subexpression $3*5$ either has type integer or complex, depending on its context. If the complete expression is $2*(3*5)$, then $3*5$ must have type integer because $*$ takes either a pair of integers or a pair of complex numbers as arguments. On the other hand, $3*5$ must have type complex if the complete expression is $(3*5)*z$ and z is declared to be complex. □

In Section 6.2, we assumed that each expression had a unique type, so the type-checking rule for function application was:

$$E \rightarrow E_1 (E_2) \quad \{ \begin{array}{l} E.type := \text{if } E_2.type = s \text{ and} \\ \quad E_1.type = s \rightarrow t \text{ then } t \\ \quad \text{else } type_error \end{array} \}$$

The natural generalization of this rule to sets of types appears in Fig. 6.10. The only operation in Fig. 6.10 is that of function application; the rules for checking other operators in expressions are similar. There may be several declarations of an overloaded identifier, so we assume that a symbol-table entry may contain a set of possible types; this set is returned by the *lookup* function. The starting nonterminal E' generates a complete expression. Its role is clarified below.

PRODUCTION	SEMANTIC RULE
$E' \rightarrow E$	$E'.types := E.types$
$E \rightarrow \text{ld}$	$E.types := \text{lookup}(\text{ld.entry})$
$E \rightarrow E_1 (E_2)$	$E.types := \{ t \mid \text{there exists an } s \text{ in } E_2.types \text{ such that } s \rightarrow t \text{ is in } E_1.types \}$

Fig. 6.10. Determining the set of possible types of an expression.

In words, the third rule of Fig. 6.10 says that if s is one of the types of E_2 and one of the types of E_1 can map s to t , then t is one of the types of $E_1(E_2)$. A type mismatch during function application results in the set $E.types$ becoming empty, a condition we temporarily use to signal a type error.

Example 6.7. Besides illustrating the specification in Fig. 6.10, this example suggests how the approach carries over to other constructs. In particular, we consider the expression $3*5$. Let the declarations of operator $*$ be as in Example 6.6. That is, $*$ can map a pair of integers to either an integer or a complex number depending on the context. The set of possible types for the subexpressions of $3*5$ is shown in Fig. 6.11, where i and c abbreviate *integer* and *complex*, respectively.

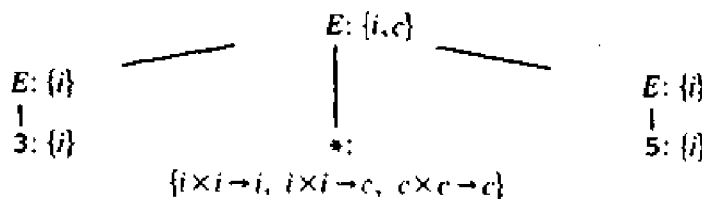


Fig. 6.11. Set of possible types for the expression $3*5$.

Again, suppose that the only possible type for 3 and 5 is *integer*. The operator $*$ is therefore applied to a pair of integers. If we treat this pair of

integers as a unit, its type is given by *integer* $\times$ *integer*. There are two functions in the set of types for $*$ that apply to pairs of integers; one returns an integer, while the other returns a complex number, so the root can have either type *integer* or type *complex*. $\square$

Narrowing the Set of Possible Types

Ada requires a complete expression to have a unique type. Given a unique type from the context, we can narrow down the type choices for each subexpression. If this process does not result in a unique type for each subexpression, then a type error is declared for the expression.

Before working top down from an expression to its subexpressions, we take a close look at the sets *E.types* constructed by the rules in Fig. 6.10. We show that every type *t* in *E.types* is a *feasible* type; i.e., it is possible to choose from among the overloaded types of the identifiers in *E* in such a way that *E* gets type *t*. The property holds for identifiers by declaration, since each element of *id.types* is feasible. For the inductive step, consider type *t* in *E.types*, where *E* is $E_1(E_2)$. From the rule for function application in Fig. 6.10, for some type *s*, *s* must be in *E₂.types* and a type $s \rightarrow t$ must be in *E₁.types*. By induction, *s* and $s \rightarrow t$ are feasible types for *E₂* and *E₁*, respectively. It follows that *t* is a feasible type for *E*.

There may be several ways of arriving at a feasible type. For example, consider the expression $f(x)$ where *f* can have the types $a \rightarrow c$ and $b \rightarrow c$, and *x* can have the types *a* and *b*. Then, $f(x)$ has type *c* but *x* can have either type *a* or *b*.

The syntax-directed definition in Fig. 6.12 is obtained from that in Fig. 6.10 by adding semantic rules to determine inherited attribute *unique* of *E*. Synthesized attribute *code* of *E* is discussed below.

Since the entire expression is generated by *E'*, we want *E'.types* to be a set containing a single type *t*. This single type is inherited as the value of *E.unique*. Again, the basic type *type_error* signals an error.

If a function $E_1(E_2)$ returns type *t*, then we can find a type *s* that is feasible for the argument *E₂*; at the same time, $s \rightarrow t$ is feasible for the function. The set *S* in the corresponding semantic rule in Fig. 6.12 is used to check that there is a unique type *s* with this property.

The syntax-directed definition in Fig. 6.12 can be implemented by making two depth-first traversals of a syntax tree for an expression. During the first traversal, the attribute *types* is synthesized bottom up. During the second traversal, the attribute *unique* is propagated top down, and as we return from a node, the *code* attribute can be synthesized. In practice, the type checker may simply attach a unique type to each node of the syntax tree. In Fig. 6.12, we generate postfix notation to suggest how intermediate code might be generated. In the postfix notation, each identifier and instance of the **apply** operator has a type attached to it by the function *gen*.

PRODUCTION	SEMANTIC RULES
$E' \rightarrow E$	$E'.types := E.types$ $E'.unique := \text{if } E'.types = \{t\} \text{ then } t \text{ else type_error}$ $E'.code := E.code$
$E \rightarrow \text{id}$	$E.types := \text{lookup}(\text{id.entry})$ $E.code := \text{gen}(\text{id.lexeme} : ' E.unique)$
$E \rightarrow E_1 (E_2)$	$E.types := \{ s' \mid \text{there exists an } s \text{ in } E_2.types$ such that $s \rightarrow s'$ is in $E_1.types\}$ $t := E.unique$ $S := \{ s \mid s \in E_2.types \text{ and } s \rightarrow t \in E_1.types \}$ $E_2.unique := \text{if } S = \{s\} \text{ then } s \text{ else type_error}$ $E_1.unique := \text{if } S = \{s\} \text{ then } s \rightarrow t \text{ else type_error}$ $E.code := E_1.code \parallel E_2.code \parallel \text{gen}(' \text{apply}' : ' E.unique)$

Fig. 6.12. Narrowing down the set of types for an expression.

6.6 POLYMORPHIC FUNCTIONS

An ordinary procedure allows the statements in its body to be executed with arguments of fixed types; each time a polymorphic procedure is called, the statements in its body can be executed with arguments of different types. The term “polymorphic” can also be applied to any piece of code that can be executed with arguments of different types, so we can talk of polymorphic functions and operators as well.

Built-in operators for indexing arrays, applying functions, and manipulating pointers are usually polymorphic because they are not restricted to a particular kind of array, function, or pointer. For example, the C reference manual states about the pointer operator `&`: “If the type of the operand is ‘...’, the type of the result is ‘pointer to ...’.” Since any type can be substituted for “...” the operator `&` in C is polymorphic.

In Ada, “generic” functions are polymorphic, but polymorphism in Ada is restricted. Since the term “generic” has also been used to refer to overloaded functions and to the coercion of the arguments of functions, we shall avoid using that term.

This section addresses the problems that arise in designing a type checker for a language with polymorphic functions. To deal with polymorphism, we extend our set of type expressions to include expressions with type variables. The inclusion of type variables raises some algorithmic issues concerning the equivalence of type expressions.

Why Polymorphic Functions?

Polymorphic functions are attractive because they facilitate the implementation of algorithms that manipulate data structures, regardless of the types of the elements in the data structure. For example, it is convenient to have a program that determines the length of a list without having to know the types of the elements on the list.

```

type link = ↑ cell;
cell = record
    info: integer;
    next: link
end;

function length ( lptr : link ) : integer;
var len : integer;
begin
    len := 0;
    while lptr <> nil do begin
        len := len + 1;
        lptr := lptr↑.next
    end;
    length := len
end;
```

Fig. 6.13. Pascal program for the length of a list.

Languages like Pascal require full specification of the types of function parameters, so a function for determining the length of a linked list of integers cannot be applied to a list of reals. The Pascal code in Fig. 6.13 is for lists of integers. The function `length` follows the next links in the list until a `nil` link is reached. Although the function does not in any way depend on the type of the information in a cell, Pascal requires the type of the `info` field to be declared when the `length` function is written.

```

fun length(lptr) =
    if null(lptr) then 0
    else length(tl(lptr)) + 1;
```

Fig. 6.14. ML program for the length of a list.

In a language with polymorphic functions, like ML (Milner[1984]), a function `length` can be written so it applies to any kind of list, as shown in Fig. 6.14. The keyword `fun` indicates that `length` is a recursive function. The functions `null` and `tl` are predefined; `null` tests if a list is empty, and `tl`

returns the remainder of the list after the first element is removed. With the definition shown in Fig. 6.14, both the following applications of the function `length` yield 3:

```
length(["sun", "mon", "tue"]);  
length([10, 9, 8]);
```

In the first, `length` is applied to a list of strings; in the second, it is applied to a list of integers.

Type Variables

Variables representing type expressions allow us to talk about unknown types. In the remainder of this section, we shall use Greek letters α , β , $\dots$ for type variables in type expressions.

An important application of type variables is checking consistent usage of identifiers in a language that does not require identifiers to be declared before they are used. A variable represents the type of an undeclared identifier. We can tell by looking at the program whether the undeclared identifier is used, say, as an integer in one statement and as an array in another. Such inconsistent usage can be reported as an error. On the other hand, if the variable is always used as an integer, then we have not only ensured consistent usage; in the process, we have inferred what its type must be.

Type inference is the problem of determining the type of a language construct from the way it is used. The term is often applied to the problem of inferring the type of a function from its body.

Example 6.8. Type-inference techniques can be applied to programs in languages like C and Pascal to fill in missing type information at compile time. The code fragment in Fig. 6.15 shows a procedure `mlist`, which has a parameter `p` that is itself a procedure. All we know by looking at the first line of the procedure `mlist` is that `p` is a procedure; in particular, we do not know the number or types of the arguments taken by `p`. Such incomplete specifications of the type of `p` are allowed by C and by the Pascal reference manual.

The procedure `mlist` applies parameter `p` to every cell in a linked list. For example, `p` may be used to initialize or print the integer held in a cell. Despite the fact that the types of the arguments of `p` are not specified, we can infer from the use of `p` in the expression `p(1ptr)` that the type of `p` must be:

`link  $\rightarrow$  void`

Any call of `mlist` with a procedure parameter that does not have this type is an error. A procedure can be thought of as a function that does not return a value, so its result type is *void*. $\square$

Techniques for type inference and type checking have a lot in common. In each case, we have to deal with type expressions containing variables. Reasoning similar to that in the following example is used later in this section by


```

type link ↑cell;

procedure mlist ( lptr : link; procedure p ) ;
begin
    while lptr <> nil do begin
        p(lptr);
        lptr := lptr↑.next
    end
end;

```

Fig. 6.15. Procedure mlist with procedure parameter p.

a type checker to infer the types represented by variables.

Example 6.9. A type can be inferred for the polymorphic function `deref` in the following pseudo-program. Function `deref` has the same effect as the Pascal operator `↑` for dereferencing pointers.

```

function deref(p);
begin
    return p↑
end;

```

When the first line

```
function deref(p);
```

is seen, we know nothing about the type of `p`, so let us represent it by a type variable β . By definition, the postfix operator `↑` takes a pointer to an object, and returns the object. Since the `↑` operator is applied to `p` in expression `p↑`, it follows that `p` must be a pointer to an object of unknown type α , so we learn that

$$\beta = \text{pointer}(\alpha)$$

where α is another type variable. Furthermore, the expression `p↑` has type α , so we can write the type expression

$$\text{for any type } \alpha, \text{ pointer}(\alpha) \rightarrow \alpha \quad (6.3)$$

for the type of the function `deref`. □

A Language with Polymorphic Functions

All we have said so far about polymorphic functions is that they can be executed with arguments of “different types.” Precise statements about the set of types to which a polymorphic function can be applied are made using the symbol $\forall$, meaning “for any type.” Thus,

$$\forall \alpha. \text{ pointer}(\alpha) \rightarrow \alpha \quad (6.4)$$

is how we write the type expression (6.3) for the type of the function `deref` in Example 6.9. The polymorphic function `length` in Fig. 6.14 takes a list of elements of any type and returns an integer, so its type can be written as:

$$\forall \alpha. \text{list}(\alpha) \rightarrow \text{integer} \quad (6.5)$$

Here, `list` is a type constructor. Without the $\forall$ symbol, we can only give examples of possible domain and range types for `length`:

$$\begin{aligned} \text{list}(\text{integer}) &\rightarrow \text{integer} \\ \text{list}(\text{list}(\text{char})) &\rightarrow \text{integer} \end{aligned}$$

Type expressions like (6.5) are the most general statements that we can make about the type of a polymorphic function.

The $\forall$ symbol is the *universal quantifier*, and the type variable to which it is applied is said to be *bound* by it. Bound variables can be renamed at will, provided all occurrences of the variable are renamed. Thus, the type expression

$$\forall \gamma. \text{pointer}(\gamma) \rightarrow \gamma$$

is equivalent to (6.4). A type expression with a $\forall$ symbol in it will be referred to informally as a “polymorphic type.”

The language we shall use for checking polymorphic functions is generated by the grammar in Fig. 6.16.

$$\begin{aligned} P &\rightarrow D ; E \\ D &\rightarrow D ; D \mid \text{id} : Q \\ Q &\rightarrow \forall \text{type_variable} . Q \mid T \\ T &\rightarrow T \rightarrow T \\ &\mid T \times T \\ &\mid \text{unary_constructor} (T) \\ &\mid \text{basic_type} \\ &\mid \text{type_variable} \\ &\mid (T) \\ E &\rightarrow E (E) \mid E , E \mid \text{id} \end{aligned}$$

Fig. 6.16. Grammar for language with polymorphic functions.

Programs generated by this grammar consist of a sequence of declarations followed by the expression `E` to be checked, for example,

$$\begin{aligned} \text{deref} &: \forall \alpha. \text{pointer}(\alpha) \rightarrow \alpha; \\ q &: \text{pointer}(\text{pointer}(\text{integer})); \\ \text{deref}(\text{deref}(q)) \end{aligned} \quad (6.6)$$

We minimize notation by having nonterminal `T` generate type expressions directly. The constructors $\rightarrow$ and $\times$ form function and product types. Unary constructors, represented by `unary_constructor`, allow types like

pointer(integer) and *list(integer)* to be written. Parentheses are used simply for grouping types. Expressions whose types are to be checked have a very simple syntax: they can be identifiers, sequences of expressions forming a tuple, or the application of a function to an argument.

The type-checking rules for polymorphic functions differ in three ways from those for ordinary functions in Section 6.2. Before presenting the rules, we illustrate these differences by considering the expression *deref(deref(q))* in the program (6.6). A syntax tree for this expression is shown in Fig. 6.17. Attached to each node are two labels. The first tells us the subexpression represented by the node and the second is a type expression assigned to the subexpression. Subscripts *o* and *i* distinguish between the outer and inner occurrences of *deref*, respectively.

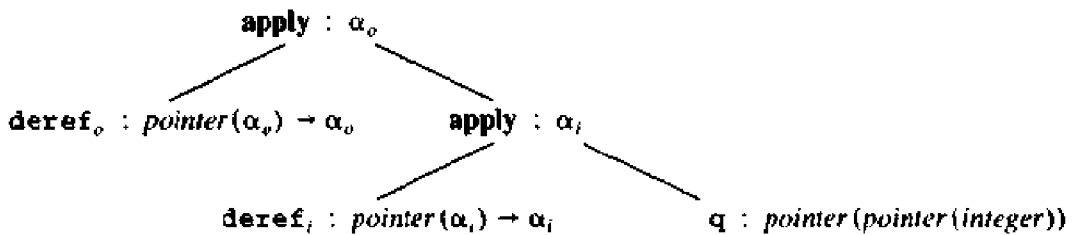


Fig. 6.17. Labeled syntax tree for *deref(deref(q))*.

The differences from the rules for ordinary functions are:

1. Distinct occurrences of a polymorphic function in the same expression need not have arguments of the same type. In the expression *deref_o(deref_i(q))*, *deref_i* removes one level of pointer indirection, so *deref_o* is applied to an argument of a different type. The implementation of this property is based on the interpretation of $\forall \alpha$ as "for any type α ." Each occurrence of *deref* has its own view of what the bound variable α in (6.4) stands for. We therefore assign to each occurrence of *deref* a type expression formed by replacing α in (6.4) by a fresh variable and removing the $\forall$ quantifier in the process. In Fig. 6.17, the fresh variables α_o and α_i are used in the type expressions assigned to the outer and inner occurrences of *deref*, respectively.
2. Since variables can appear in type expressions, we have to reexamine the notion of equivalence of types. Suppose E_1 of type $s \rightarrow s'$ is applied to E_2 of type t . Instead of simply determining the equivalence of s and t , we must "unify" them. Unification is defined below; informally, we determine if s and t can be made structurally equivalent by replacing the type variables in s and t by type expressions. For example, at the inner node labeled **apply** in Fig. 6.17, the equality

$$\text{pointer}(\alpha_i) = \text{pointer}(\text{pointer}(\text{integer}))$$

is true if α_i is replaced by *pointer(integer)*.

3. We need a mechanism for recording the effect of unifying two expressions. In general, a type variable may appear in several type expressions. If unification of s and s' results in variable α representing type t , then α must continue to represent t as type checking proceeds. For example, in Fig. 6.17, α_i is the range type of deref_i , so we can use it for the type of $\text{deref}_i(q)$. Unifying the domain type of deref_i with the type of q therefore affects the type expression at the inner node labeled **apply**. The other type variable α_o in Fig. 6.17 represents *integer*.

Substitutions, Instances, and Unification

Information about the types represented by variables is formalized by defining a mapping from type variables to type expressions called a *substitution*. The following recursive function *subst*(t) makes precise the notion of applying a substitution S to replace all type variables in an expression t . As usual, we take the function type constructor to be the “typical” constructor.

```
function subst ( $t$  : type_expression) : type_expression;
begin
    if  $t$  is a basic type then return  $t$ 
    else if  $t$  is a variable then return  $S(t)$ 
    else if  $t$  is  $t_1 \rightarrow t_2$  then return subst( $t_1$ )  $\rightarrow$  subst( $t_2$ )
end
```

For convenience, we write $S(t)$ for the type expression that results when *subst* is applied to t ; the result $S(t)$ is called an *instance* of t . If substitution S does not specify an expression for variable α , we assume that $S(\alpha)$ is α ; that is, S is the identity mapping on such variables.

Example 6.10. In the following, we write $s < t$ to indicate that s is an instance of t :

```
pointer(integer) < pointer( $\alpha$ )
pointer(real) < pointer( $\alpha$ )
integer  $\rightarrow$  integer <  $\alpha \rightarrow \alpha$ 
pointer( $\alpha$ ) <  $\beta$ 
 $\alpha$  <  $\beta$ 
```

However, in the following, the type expression on the left is not an instance of the one on the right (for the reason indicated):

<i>integer</i>	<i>real</i>	Substitutions do not apply to basic types.
<i>integer</i> $\rightarrow$ <i>real</i>	$\alpha \rightarrow \alpha$	Inconsistent substitution for α .
<i>integer</i> $\rightarrow \alpha$	$\alpha \rightarrow \alpha$	All occurrences of α must be replaced. □

Two type expressions t_1 and t_2 *unify* if there exists some substitution S such that $S(t_1) = S(t_2)$. In practice, we are interested in the *most general unifier*,

which is a substitution that imposes the fewest constraints on the variables in the expressions. More precisely, the most general unifier of expressions t_1 and t_2 is a substitution S with the following properties:

1. $S(t_1) = S(t_2)$ and
2. for any other substitution S' such that $S'(t_1) = S'(t_2)$, the substitution S' is an instance of S (that is, for any t , $S'(t)$ is an instance of $S(t)$).

In what follows, when we say “unifies” we are referring to the most general unifier.

Checking Polymorphic Functions

The rules for checking expressions generated by the grammar in Fig. 6.16 will be written in terms of the following operations on a graph representation of types.

1. *fresh*(t) replaces the bound variables in type expression t by fresh variables and returns a pointer to a node representing the resulting type expression. Any $\forall$ symbols in t are removed in the process.
2. *unify*(m, n) unifies the type expressions represented by the nodes pointed to by m and n . It has the side effect of keeping track of the substitution that makes the expressions equivalent. If the expressions fail to unify, the entire type-checking process fails.⁴

Individual leaves and interior nodes in the type graph are constructed using operations *mkleaf* and *mknnode* similar to those of Section 5.2. It is necessary that there be a unique leaf for each type variable, but other structurally equivalent expressions need not have unique nodes.

The *unify* operation is based on the following graph-theoretic formulation of unification and substitutions. Suppose nodes m and n of a graph represent expressions e and f , respectively. We say nodes m and n are *equivalent under* substitution S if $S(e) = S(f)$. The problem of finding the most general unifier S can be restated as the problem of grouping, into sets, nodes that must be equivalent under S . For the expressions to be equivalent, their roots must be equivalent. Also, two nodes m and n are equivalent if and only if they represent the same operator and their corresponding children are equivalent.

An algorithm for unifying a pair of expressions is deferred until the next section. The algorithm keeps track of sets of nodes that are equivalent under the substitutions that have occurred.

The type-checking rules for expressions are shown in Fig. 6.18. We do not show how declarations are processed. As type expressions generated by

⁴ The reason for aborting the type-checking process is that the side effects of some unifications may be recorded before failure is detected. Error recovery can be implemented if the side effects of the *unify* operation are deferred until the expressions have been unified successfully.

nonterminals T and Q are examined, $mkleaf$ and $mknnode$ add nodes to the type graph, following the dag construction in Section 5.2. When an identifier is declared, the type in the declaration is saved in the symbol table in the form of a pointer to the node representing the type. In Fig. 6.18, this pointer is referred to as the synthesized attribute $id.type$. As mentioned above, the *fresh* operation removes the $\forall$ symbols as it replaces bound variables by fresh variables. The action associated with production $E \rightarrow E_1, E_2$ sets $E.type$ to the product of the types of E_1 and E_2 .

$$\begin{aligned} E \rightarrow E_1 (E_2) & \quad \{ p := mkleaf(newtypevar); \\ & \quad \quad \quad unify(E_1.type, mknnode(' \rightarrow ', E_2.type, p)); \\ & \quad \quad \quad E.type := p \} \\ E \rightarrow E_1 , E_2 & \quad \{ E.type := mknnode(' \times ', E_1.type, E_2.type) \} \\ E \rightarrow id & \quad \{ E.type := fresh(id.type) \} \end{aligned}$$

Fig. 6.18. Translation scheme for checking polymorphic functions.

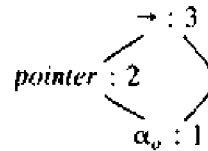
The type-checking rule for the function application $E \rightarrow E_1(E_2)$ is motivated by considering the case where $E_1.type$ and $E_2.type$ are both type variables, say, $E_1.type = \alpha$ and $E_2.type = \beta$. Here $E_1.type$ must be a function such that for some unknown type γ , we have $\alpha = \beta \rightarrow \gamma$. In Fig. 6.18, a fresh type variable corresponding to γ is created and $E_1.type$ is unified with $E_2.type \rightarrow \gamma$. A new type variable is returned by each call of *newtypevar*, a leaf for it is constructed by *mkleaf*, and a node representing the function to be unified with $E_1.type$ is constructed by *mknnode*. After unification succeeds, the new leaf represents the result type.

The rules in Fig. 6.18 will be illustrated by working out a simple example in detail. We summarize the workings of the algorithm by writing down the type expression assigned to each subexpression, as in Fig. 6.19. At each function application, the *unify* operation may have the side effect of recording a type expression for some of the type variables. Such side effects are suggested by the column for a substitution in Fig. 6.19.

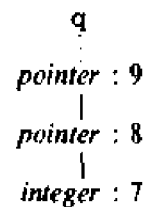
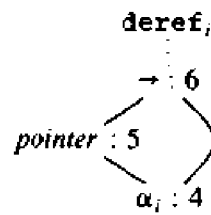
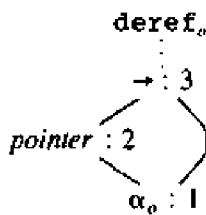
EXPRESSION : TYPE	SUBSTITUTION
$q : pointer(pointer(integer))$	
$deref_i : pointer(\alpha_i) \rightarrow \alpha_i$	
$deref_i(q) : pointer(integer)$	$\alpha_i = pointer(integer)$
$deref_o : pointer(\alpha_o) \rightarrow \alpha_o$	
$deref_o(deref_i(q)) : integer$	$\alpha_o = integer$

Fig. 6.19. Summary of bottom-up type determination.

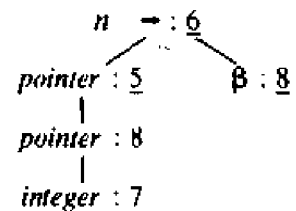
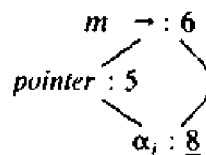
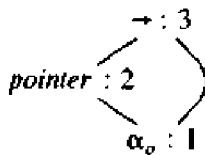
Example 6.11. Type checking the expression $\text{deref}_o(\text{deref}_i(q))$ in program (6.6) proceeds bottom up from the leaves. Once again, subscripts o and i distinguish between occurrences of deref . When subexpression deref_o is considered, *fresh* constructs the following nodes using a new type variable α_o .



The number at a node indicates the equivalence class the node belongs to. The part of the type graph for the three identifiers is shown below. The dotted lines indicate that nodes numbered 3, 6, and 9 are for deref_o , deref_i , and q , respectively.



The function application $\text{deref}_i(q)$ is checked by constructing a node n for a function from the type of q to a new type variable β . This function unifies successfully with the type of deref_i represented by node m below. Before nodes m and n are unified, each node had a distinct number. After unification, the equivalent nodes are the ones below with the same number; the changed numbers are underlined:



Note that the node for α_i and $\text{pointer}(\text{integer})$ are both numbered 8, that is α_i is unified with this type expression, as shown in Fig. 6.19. Subsequently, α_o is unified with integer . $\square$

The next example relates type inference of polymorphic functions in ML to the type-checking rules in Fig. 6.18. The syntax of function definitions in ML is given by

fun id_0 ($\text{id}_1, \dots, \text{id}_k$) = E ;

where id_0 represents the function name and $\text{id}_1, \dots, \text{id}_k$ represent its parameters. For simplicity, we assume that the syntax of expression E is as in Fig. 6.16, and that the only identifiers in E are the function name, its parameters,

and built-in functions.

The approach is a formalization of that in Example 6.9, where a polymorphic type was inferred for `deref`. New type variables are made up for the function name and its parameters. The built-in functions have polymorphic types, in general; any type variables that appear in these types are bound by $\forall$ quantifiers. We then check that the types of the expressions $id_0(id_1, \dots, id_k)$ and E match. When the match succeeds, we will have inferred a type for the function name. Finally, any variables in the inferred type are bound by $\forall$ quantifiers to give the polymorphic type for the function.

Example 6.12. Recall the ML function in Fig. 6.14 for determining the length of a list:

```
fun length(lptr) =
    if null(lptr) then 0
    else length(tl(lptr)) + 1;
```

Type variables β and γ are introduced for the types of `length` and `lptr`, respectively. We find that the type of `length(lptr)` matches that of the expression forming the function body and that `length` must have type

for any type α , $list(\alpha) \rightarrow integer$

so the type of `length` is

$\forall \alpha. list(\alpha) \rightarrow integer$

In more detail, we set up the program shown in Fig. 6.20, to which the type-checking rules of Fig. 6.18 can be applied. The declarations in the program associate the new type variables β and γ with `length` and `lptr`, and make explicit the types of the built-in operations. We write conditionals in

```
length :  $\beta$ ;
lptr :  $\gamma$ ;
if :  $\forall \alpha. boolean \times \alpha \times \alpha \rightarrow \alpha$ ;
null :  $\forall \alpha. list(\alpha) \rightarrow boolean$ ;
tl :  $\forall \alpha. list(\alpha) \rightarrow list(\alpha)$ ;
0 : integer;
1 : integer;
+ :  $integer \times integer \rightarrow integer$ ;
match :  $\forall \alpha. \alpha \times \alpha \rightarrow \alpha$ ;

match(
    length(lptr),
    if( null(lptr), 0, length(tl(lptr)) + 1 )
)
```

Fig. 6.20. Declarations followed by expression to be checked.

LINE	EXPRESSION : TYPE	SUBSTITUTION
(1)	$lptr : \gamma$	
(2)	$length : \beta$	
(3)	$length(lptr) : \delta$	$\beta = \gamma \rightarrow \delta$
(4)	$lptr : \gamma$	
(5)	$null : list(\alpha_n) \rightarrow boolean$	
(6)	$null(lptr) : boolean$	$\gamma = list(\alpha_n)$
(7)	$0 : integer$	
(8)	$lptr : list(\alpha_n)$	
(9)	$tl : list(\alpha_t) \rightarrow list(\alpha_t)$	
(10)	$tl(lptr) : list(\alpha_n)$	$\alpha_t = \alpha_n$
(11)	$length : list(\alpha_n) \rightarrow \delta$	
(12)	$length(tl(lptr)) : \delta$	
(13)	$1 : integer$	
(14)	$+ : integer \times integer \rightarrow integer$	
(15)	$length(tl(lptr))+1 : integer$	$\delta = integer$
(16)	$if : boolean \times \alpha_1 \times \alpha_1 \rightarrow \alpha_1$	
(17)	$if(\dots) : integer$	$\alpha_1 = integer$
(18)	$match : \alpha_n \times \alpha_n \rightarrow \alpha_n$	
(19)	$match(\dots) : integer$	$\alpha_n = integer$

Fig. 6.21. Inferring the type $list(\alpha_n) \rightarrow integer$ for $length$.

the style of Fig. 6.16 by applying polymorphic operator *if* to three operands, representing the expression to be tested, the then part, and the else part; the declaration says that the then and else parts can be of any matching type, which is also the type of the result.

Clearly, $length(lptr)$ must have the same type as the function body; this check is encoded using an operator *match*. The use of *match* is a technical convenience that allows all checking to be done using a program in the style of Fig. 6.16.

The effect of applying the type-checking rules in Fig. 6.18 to the program in Fig. 6.20 is summarized in Fig. 6.21. The new variables introduced by operation *fresh* applied to the polymorphic types of the built-in operations are distinguished by subscripts on α . We learn on line (3) that $length$ must be a function from γ to some unknown type δ . Then, when the subexpression $null(lptr)$ is checked, we find on line (6) that γ unifies with $list(\alpha_n)$, where α_n is an unknown type. At this point, we know that the type of $length$ must be

for any type α_n , $list(\alpha_n) \rightarrow \delta$

Eventually, when the addition is checked on line (15) — we take the liberty of writing $+$ between its arguments for clarity — δ is unified with *integer*.

When checking is complete, the type variable α_n remains in the type of `length`. Since no assumptions were made about α_n , any type can be substituted for it when the function is used. We therefore make it a bound variable and write

$$\forall \alpha_n. \text{list}(\alpha_n) \rightarrow \text{integer}$$

for the type of `length`. □

6.7 AN ALGORITHM FOR UNIFICATION

Informally, unification is the problem of determining whether two expressions e and f can be made identical by substituting expressions for the variables in e and f . Testing equality of expressions is a special case of unification; if e and f have constants but no variables, then e and f unify if and only if they are identical. The unification algorithm in this section can be applied to graphs with cycles, so it can be used to test structural equivalence of circular types.⁵

Unification was defined in the last section in terms of a function S , called a substitution, mapping variables to expressions. We write $S(e)$ for the expression obtained when each variable α in e is replaced by $S(\alpha)$. S is a unifier for e and f if $S(e) = S(f)$. The algorithm in this section determines a substitution that is the most general unifier of a pair of expressions.

Example 6.13. For a perspective on most general unifiers, consider the two type expressions

$$\begin{aligned} ((\alpha_1 \rightarrow \alpha_2) \times \text{list}(\alpha_3)) &\rightarrow \text{list}(\alpha_2) \\ ((\alpha_3 \rightarrow \alpha_4) \times \text{list}(\alpha_3)) &\rightarrow \alpha_5 \end{aligned}$$

Two unifiers, S and S' , for these expressions are:

x	$S(x)$	$S'(x)$
α_1	α_3	α_1
α_2	α_2	α_1
α_3	α_3	α_1
α_4	α_2	α_1
α_5	$\text{list}(\alpha_2)$	$\text{list}(\alpha_1)$

These substitutions map e and f as follows:

$$\begin{aligned} S(e) &= S(f) = ((\alpha_3 \rightarrow \alpha_2) \times \text{list}(\alpha_3)) \rightarrow \text{list}(\alpha_2) \\ S'(e) &= S'(f) = ((\alpha_1 \rightarrow \alpha_1) \times \text{list}(\alpha_1)) \rightarrow \text{list}(\alpha_1) \end{aligned}$$

Substitution S is the most general unifier of e and f . Note that $S'(e)$ is an instance of $S(e)$ because we can substitute α_1 for both variables in $S(e)$.

⁵ In some applications, it is an error to unify a variable with an expression containing that variable. Algorithm 6.1 permits such substitutions.

However, the reverse is false, because the same expression must be substituted for each occurrence of α_1 in $S'(e)$, so we cannot obtain $S(e)$ by substituting for the variable α_1 in $S'(e)$. $\square$

When expressions to be unified are represented by trees, the number of nodes in the tree for the substituted expression $S(e)$ can be exponential in the number of nodes in the trees for e and f , even if S is the most general unifier. However, such a size blowup need not occur if graphs rather than trees are used to represent expressions and substitutions.

We shall implement the graph-theoretic formulation of unification, also presented in the last section. The problem is that of grouping, into sets, nodes that must be equivalent under the most general unifier of two expressions. The two expressions in Example 6.13 are represented by the two nodes labeled $\rightarrow:1$ in Fig. 6.22. The integers at the nodes indicate the equivalence classes that the nodes belong to after the nodes numbered 1 are unified. These equivalence classes have the property that all interior nodes in the class are for the same operator. The corresponding children of interior nodes in an equivalence class are also equivalent.

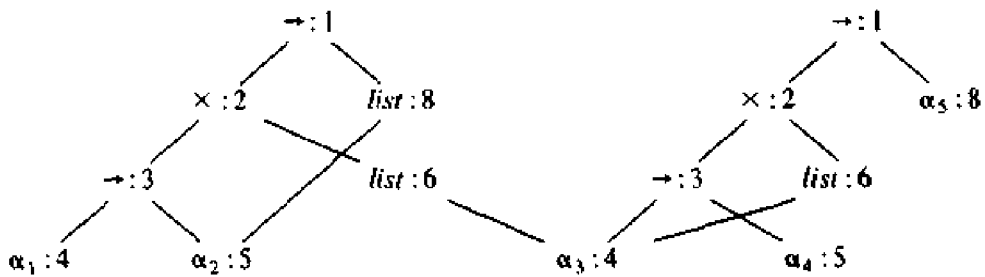


Fig. 6.22. Equivalence classes after unification.

Algorithm 6.1. Unification of a pair of nodes in a graph.

Input. A graph and a pair of nodes m and n to be unified.

Output. Boolean value true if the expressions represented by the nodes m and n unify; false, otherwise. The version of operation *unify* needed for the type-checking rules of Fig. 6.18 is obtained if the function in this algorithm is modified to fail instead of returning false.

Method. A node is represented by a record as in Fig. 6.23 with fields for a binary operator and pointers to the left and right children.

The sets of equivalent nodes are maintained using the *set* field. One node in each equivalence class is chosen the unique representative of the equivalence class by making its *set* field contain a nil pointer. The *set* fields of the remaining nodes in the equivalence class will point (possibly indirectly through other nodes in the set) to the representative. Initially, each node n is in an equivalence class by itself, with n as its own representative node.

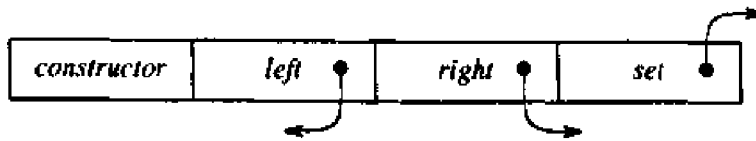


Fig. 6.23. Data structure for a node.

```

function unify (m, n : node) : boolean
begin
    s := find(m);
    t := find(n);
    if s = t then
        return true
    else if s and t are nodes that represent the same basic type then
        return true
    else if s is an op-node with children s1 and s2 and
        t is an op-node with children t1 and t2 then begin
        union(s, t);
        return unify(s1, t1) and unify(s2, t2)
    end
    else if s or t represents a variable then begin
        union(s, t);
        return true
    end
    else return false
    /* interior nodes with different operators cannot be unified */
end

```

Fig. 6.24. Unification algorithm.

The unification algorithm, shown in Fig. 6.24, uses the following two operations on nodes:

1. *find*(*n*) returns the representative node of the equivalence class currently containing node *n*.
2. *union*(*m*, *n*) merges the equivalence classes containing nodes *m* and *n*. If one of the representatives for the equivalence classes of *m* and *n* is a non-variable node, *union* makes that non-variable node be the representative for the merged equivalence class; otherwise, *union* makes one or the other of the original representatives be the new representative. This asymmetry in the specification of *union* is important because a variable cannot be used as the representative for an equivalence class for an

expression containing a type constructor or basic type. Otherwise, two inequivalent expressions may be unified through that variable.

The *union* operation on sets is implemented by simply changing the *set* field of the representative of one equivalence class so that it points to the representative of the other. To find the equivalence class that a node belongs to, we follow the *set* pointers of nodes until the representative (the node with a *nil* pointer in the *set* field) is reached.

Note that the algorithm in Fig. 6.24 uses $s = \text{find}(m)$ and $t = \text{find}(n)$ rather than m and n , respectively. The representative nodes s and t are equal if m and n are in the same equivalence class. If s and t represent the same basic type, the call $\text{unify}(m, n)$ returns true. If s and t are both interior nodes for a binary type constructor, we merge their equivalence classes on speculation and recursively check that their respective children are equivalent. By merging first, we decrease the number of equivalence classes before recursively checking the children, so the algorithm terminates.

The substitution of an expression for a variable is implemented by adding the leaf for the variable to the equivalence class containing the node for the expression. If either m or n is a leaf for a variable that has been put into an equivalence class that contains a node representing an expression with a type constructor or a basic type, then find will return a representative that reflects that type constructor or basic type, so that a variable cannot be unified with two different expressions. $\square$

Example 6.14. We have shown the initial graph for the two expressions of Example 6.13 in Fig. 6.25 with each node numbered and in its own equivalence class. To compute $\text{unify}(1, 9)$, the algorithm notes that nodes 1 and 9 both represent the same operator so it merges 1 and 9 into the same equivalence class and calls $\text{unify}(2, 10)$ and $\text{unify}(8, 14)$. The result of computing $\text{unify}(1, 9)$ is the graph previously shown in Fig. 6.22. $\square$

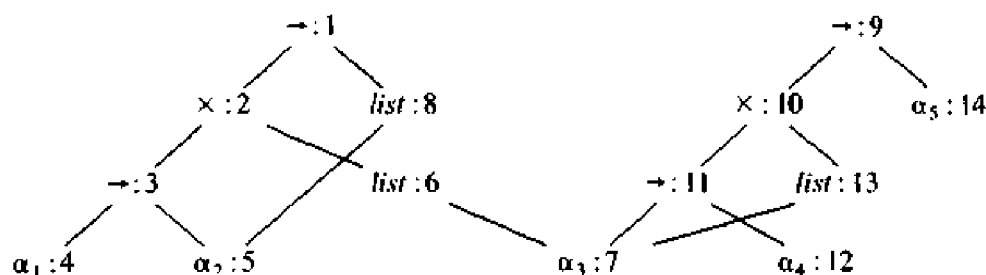


Fig. 6.25. Initial dag with each node in its own equivalence class.

If Algorithm 6.1 returns true, we can construct a substitution S that acts as the unifier, as follows. Let each node n of the resulting graph represent the expression associated with $\text{find}(n)$. Thus, for each variable α , $\text{find}(\alpha)$ gives the node n that is the representative of the equivalence class of α . The

expression represented by n is $S(\alpha)$. For example, in Fig. 6.22, we see that the representative for α_3 is node 4, which represents α_1 . The representative for α_5 is node 8, which represents $list(\alpha_2)$.

Example 6.15. Algorithm 6.1 can be used to test the structural equivalence of the two type expressions

$$\begin{aligned} e &: real \rightarrow e \\ f &: real \rightarrow (real \rightarrow \epsilon) \end{aligned}$$

The type graphs for these expressions are shown in Fig. 6.26. For convenience, each node has been numbered.

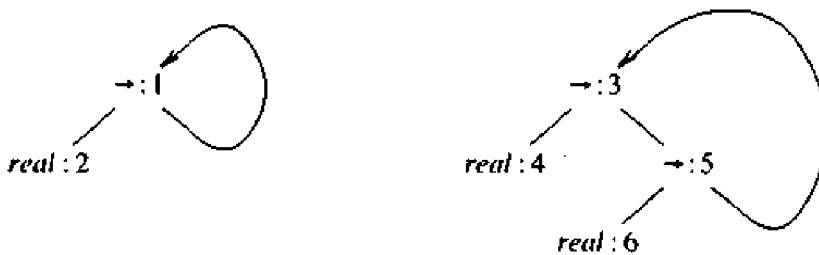


Fig. 6.26. Graph for two circular types.

We call $unify(1, 3)$ to test for the structural equivalence of these two expressions. The algorithm merges nodes 1 and 3 into one equivalence class, and recursively calls $unify(2, 4)$ and $unify(1, 5)$. Since 2 and 4 represent the same basic type, the call $unify(2, 4)$ returns true. The call $unify(1, 5)$ adds 5 to the equivalence class of 1 and 3, and recursively calls $unify(2, 6)$ and $unify(1, 3)$.

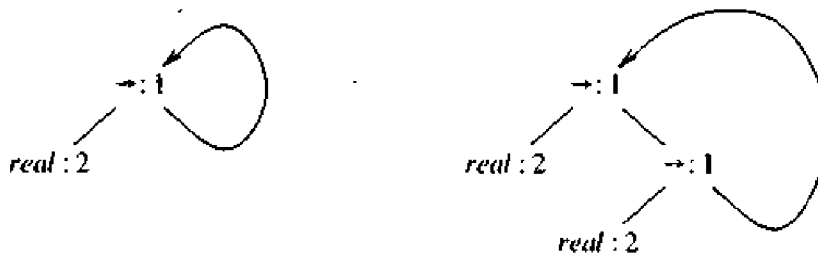


Fig. 6.27. Type graph showing equivalence classes of nodes.

The call $unify(2, 6)$ returns true because 2 and 6 also represent the same basic type. The second call of $unify(1, 3)$ terminates because we have already merged nodes 1 and 3 into the same equivalence class. The algorithm then terminates, returning true to show that the two type expressions are indeed equivalent. Figure 6.27 shows the resulting equivalence classes of nodes, where nodes with the same integer are in the same equivalence class. $\square$

EXERCISES

6.1 Write type expressions for the following types.

- a) An array of pointers to reals, where the array index ranges from 1 to 100.
- b) A two-dimensional array of integers (i.e., an array of arrays) whose rows are indexed from 0 to 9 and whose columns are indexed from -10 to 10.
- c) Functions whose domains are functions from integers to pointers to integers and whose ranges are records consisting of an integer and a character.

6.2 Suppose we have the following C declarations:

```
typedef struct {
    int a, b;
} CELL, *PCELL;
CELL foo[100];
PCELL bar(x, y) int x; CELL y { ... }
```

Write type expressions for the types of `foo` and `bar`.

6.3 The following grammar defines lists of lists of literals. The interpretation of symbols is the same as that for the grammar of Fig. 6.3 with the addition of the type `list`, which indicates a list of elements of the type `T` that follows.

$$\begin{aligned}
 P &\rightarrow D ; E \\
 D &\rightarrow D ; D \mid \text{id} : T \\
 T &\rightarrow \text{list of } T \mid \text{char} \mid \text{integer} \\
 E &\rightarrow (L) \mid \text{literal} \mid \text{num} \mid \text{id} \\
 L &\rightarrow E , L \mid E
 \end{aligned}$$

Write translation rules similar to those in Section 6.2 to determine the types of expressions (`E`) and lists (`L`).

6.4 Add to the grammar of Exercise 6.3 the production

$$E \rightarrow \text{nil}$$

meaning that an expression can be the null (empty) list. Revise the rules in your answer to Exercise 6.2 to take account of the fact that `nil` can stand for an empty list of elements of any type.

6.5 Using the translation scheme of Section 6.2, compute the types of the expressions in the following program fragments. Show the types at each node of the parse tree.

- a) `c: char; i: integer;`
`c mod i mod 3`

```

b) p: integer; a: array [10] of integer;
   a[p]

c) f: integer → boolean;
   i: integer; j: integer; k: integer;
   while f(i) do
       k := i;
       i := j mod i;
       j := k

```

- 6.6** Modify the translation scheme for checking expressions in Section 6.2 to print a descriptive message when an error is detected and to continue checking as if the expected type had been seen.
- 6.7** Rewrite the type-checking rules for expressions in Section 6.2 so they refer to nodes in a graph representation of type expressions. The rewritten rules should use data structures and operations supported by a language such as Pascal. Use structural equivalence of type expressions when:
- type expressions are represented by trees, as in Fig. 6.2, and
 - the type graph is a dag with a unique node for each type expression.
- 6.8** Modify the translation scheme of Fig. 6.5 to handle the following.
- Statements that have values. The value of an assignment is the value of the expression on the right of the `:=` sign. The value of a conditional or while statement is the value of the statement body; the value of a list of statements is the value of the last statement in the list.
 - Boolean expressions. Add productions for the logical operators **and**, **or**, and **not**, and for comparison operators (`<`, etc.). Then add appropriate translation rules giving the type of these expressions.
- 6.9** Generalize the type-checking rules for functions given at the end of Section 6.2 to handle n -ary functions.
- 6.10** Assume the type names `link` and `cell` are defined as in Section 6.3. Which among the following expressions are structurally equivalent? Which are name equivalent?
- `link`.
 - `pointer(cell)`.
 - `pointer(link)`.
 - `pointer(record((info × integer) × (next × pointer(cell))))`
- 6.11** Restate the algorithm for testing structural equivalence in Fig. 6.6 so that the arguments of *sequiv* are pointers to nodes in a dag.

- 6.12** Consider the encoding of restricted type expressions as sequences of bits in Example 6.1. In Johnson [1979], the two-bit fields for constructors appeared in the opposite order, with the field for the outermost constructor appearing next to the four bits for the basic type; for example,

TYPE EXPRESSION	ENCODING
<i>char</i>	000000 0001
<i>freturns(char)</i>	000011 0001
<i>pointer(freturns(char))</i>	001101 0001
<i>array(pointer(freturns(char)))</i>	110110 0001

Using the operators of C write code to construct the representation of *array(t)* from that of *t* and vice versa, assuming that the encoding is as in:

- a) Johnson [1979].
 - b) Example 6.1.
- 6.13** Suppose that the type of each identifier is a subrange of integers. For expressions with the operators $+$, $-$, $*$, div , and mod , as in Pascal, write type-checking rules that assign to each subexpression the subrange its value must lie in.
- 6.14** Give an algorithm to test the equivalence of C types (see Example 6.4).
- 6.15** Some languages, like PL/I, will coerce a boolean value into an integer, with true identified with 1 and false with 0. For example, $3 < 4 < 5$ is grouped $(3 < 4) < 5$, and has the value "true" (or 1), because $3 < 4$ has the value 1, and $1 < 5$ is true. Write translation rules for boolean expressions that perform this coercion. Use conditional statements in the intermediate language to assign integer values to temporaries that represent the value of a boolean expression, when needed.
- 6.16** Generalize the algorithms of (a) Fig. 6.9 and (b) Fig. 6.12 to expressions with the type constructors *array*, *pointer*, and Cartesian product.
- 6.17** Which of the following recursive type expressions are equivalent?
- $e1 = \text{integer} \rightarrow e1$
 - $e2 = \text{integer} \rightarrow (\text{integer} \rightarrow e2)$
 - $e3 = \text{integer} \rightarrow (\text{integer} \rightarrow e1)$
- 6.18** Using the rules of Example 6.6, determine which of the following expressions have unique types. Assume z is a complex number.
- a) $1 * 2 * 3$
 - b) $1 * (z * 2)$

c) $(1 * z) * z$

6.19 Suppose we allow the type conversions of Example 6.6. Under what conditions on the types of a , b , and c (integer or complex) will the expression $(a * b) * c$ have a unique type?

6.20 Express, using type variables, the types of the following functions.

- The function *ref* that takes as argument an object of any type and returns a pointer to that object.
- A function that takes as argument an array indexed by integers, with elements of any type, and returns an array whose elements are the objects pointed to by the elements of the given array.

6.21 Find the most general unifier of the type expressions

- $(\text{pointer}(\alpha)) \times (\beta \rightarrow \gamma)$
- $\beta \times (\gamma \rightarrow \delta)$

What if δ in (ii) were α ?

6.22 Find the most general unifier for each pair of expressions from the following list, or determine that none exists.

- $\alpha_1 \rightarrow (\alpha_2 \rightarrow \alpha_1)$
- $\text{array}(\beta_1) \rightarrow (\text{pointer}(\beta_1) \rightarrow \beta_3)$
- $\gamma_1 \rightarrow \gamma_2$
- $\delta_1 \rightarrow (\delta_1 \rightarrow \delta_2)$

6.23 Extend the type-checking rules in Example 6.6 to cover records. Use the following additional syntax for type expressions and expressions:

$$\begin{aligned} T &\rightarrow \text{record fields end} \\ E &\rightarrow E . \text{id} \\ \text{fields} &\rightarrow \text{fields} ; \text{field} \mid \text{field} \\ \text{field} &\rightarrow \text{id} : T \end{aligned}$$

What restrictions does the lack of type names impose on the types that can be defined?

***6.24** The resolution of overloading in Section 6.5 proceeds in two phases: first the set of possible types for each subexpression is determined and then narrowed down in a second phase to a single type after the unique type of the entire expression is determined. What data structures would you use to resolve overloading in a single bottom-up pass?

****6.25** The resolution of overloading becomes more difficult if identifier declarations are optional. More precisely, suppose that declarations can be used to overload identifiers representing function symbols, but that all occurrences of an undeclared identifier have the same type. Show that the problem of determining if an expression in this language has a valid type is NP-complete. This problem arises during

type checking in the experimental language Hope (Burstall, MacQueen, and Sannella [1980]).

- 6.26** Following Example 6.12, infer the following polymorphic type for `map`:

$$\text{map} : \forall \alpha. \forall \beta. ((\alpha \rightarrow \beta) \times \text{list}(\alpha)) \rightarrow \text{list}(\beta)$$

The ML definition of `map` is:

```
fun map(f, l) =
    if null(l) then nil
    else cons( f(hd(l)), map(f, tl(l)) )
```

The types of built-in identifiers in the function body are:

```
null :     $\forall \alpha. \text{list}(\alpha) \rightarrow \text{boolean};$ 
nil  :     $\forall \alpha. \text{list}(\alpha);$ 
cons :     $\forall \alpha. (\alpha \times \text{list}(\alpha)) \rightarrow \text{list}(\alpha);$ 
hd   :     $\forall \alpha. \text{list}(\alpha) \rightarrow \alpha;$ 
tl   :     $\forall \alpha. \text{list}(\alpha) \rightarrow \text{list}(\alpha);$ 
```

- **6.27** Show that the unification algorithm of Section 6.7 determines the most general unifier.
- *6.28** Modify the unification algorithm of Section 6.7 so that it does not unify a variable with an expression containing that variable.
- **6.29** Suppose expressions are represented by trees. Find expressions e and f such that for any unifier S , the number of nodes in $S(e)$ is exponential in the number of nodes in e and f .
- 6.30** Two nodes are said to be *congruent* if they represent equivalent expressions. Even if no two nodes in the original type graph are congruent, after unification, it is possible for distinct nodes to be congruent.
- Give an algorithm to merge a class of mutually congruent nodes into a single node.
- **b)** Extend the algorithm in (a) to merge congruent nodes until no two distinct nodes are congruent.
- *6.31** The expression `g(g)` on line 9 in the complete C program in Fig. 6.28 is the application of a function to itself. The declaration on line 3 gives *integer* as the range type `g`, but the types of the arguments of `g` is not specified. Try running the program. The compiler may issue a warning because `g` is declared to be a function on line 3, instead of a pointer to a function.
- What can you say about the type of `g`?

```

(1) int n;
(2) int f(g)
(3) int g();
(4) {
(5)     int m;
(6)     m = n;
(7)     if( m == 0 ) return 1;
(8)     else {
(9)         n = n - 1; return m * g(g);
(10)    }
(11) }
(12) main()
(13) {
(14)     n = 5; printf("%d factorial is %d\n", n, f(f) );
(15) }

```

Fig. 6.28. A C program containing applications of a function to itself.

- b) Use the type-checking rules for polymorphic functions in Fig. 6.18 to infer a type for g in the following program.

```

    m : integer;
    times : integer  $\times$  integer  $\rightarrow$  integer;
    g :  $\alpha$ ;

    times( m, g(g) )

```

BIBLIOGRAPHIC NOTES

The basic types and type constructors of early languages like Fortran and Algol 60 were limited enough that type-checking was not a serious problem. As a result, descriptions of type checking in their compilers are buried in discussions of code generation for expressions. Sheridan [1959] describes the translation of expressions in the original Fortran compiler. The compiler kept track of whether the type of an expression was integer or real, but coercions were not permitted by the language. Backus [1981, p. 54] recalls, "I think just because we didn't like the rules as to what happened with mixed mode expressions, so we decided, 'Let's throw it out. It's easier.' " Naur [1965] is an early paper on type checking in an Algol compiler; the techniques used by the compiler are similar to those discussed in Section 6.2.

Data-structuring facilities such as arrays and records were anticipated in the 1940's by Zuse in his Plankalkül that had little direct influence (Bauer and Wössner [1972]). One of the first programming languages to allow type expressions to be constructed systematically is Algol 68. Type expressions can

be recursively defined, and structural equivalence is used. A clear distinction between name and structural equivalence is found in ELI, the choice being left to the programmer (Wegbreit [1974]). The critique of Pascal by Welsh, Sneeringer, and Hoare [1977] drew attention to the distinction.

The combination of coercion and overloading can lead to ambiguities: coercing an argument may lead to overloading being resolved in favor of a different algorithm. Restrictions are therefore placed on one or the other. A free-wheeling approach to coercions was taken in PL/I, where the first design criterion was, "*Anything goes*. If a particular combination of symbols has a reasonably sensible meaning, that meaning will be made official (Radin and Rogoway [1965])." An ordering is often placed on the set of basic types — e.g., Hext [1967] describes a lattice structure imposed on the basic types of CPL — and lower types may be coerced to higher types.

Compile-time resolution of overloading in languages such as APL (Iverson [1962]) and SETL (Schwartz [1973]) has the potential for improving the running time of programs (Bauer and Saal [1974]). Tennenbaum [1974] distinguished between "forward" resolution that determines the set of possible types of an operator from its operands and "backward" resolution based on the type expected by the context. Using a lattice of types, Jones and Muchnick [1976] and Kaplan and Ullman [1980] solve constraints on types obtained from forward and backward analysis. Overloading in Ada can be resolved by making a single forward and then a single backward pass, as in Section 6.5. This observation emerges from a number of papers: Ganzinger and Ripken [1980]; Pennello, DeRemer and Meyers [1980]; Janas [1980]; Persch et al. [1980]. Cormack [1981] offers a recursive implementation and Baker [1982] avoids an explicit backward pass by carrying along a dag of possible types.

Type inference was studied by Curry (see Curry and Feys [1958]) in connection with combinatory logic and the lambda calculus of Church [1941]. It has long been observed that the lambda calculus is at the core of a functional language. We have repeatedly used the application of a function to an argument to discuss type-checking concepts in this chapter. Functions can be defined and applied with no regard to types in the lambda calculus, and Curry was interested in their "functional character," and determining what we would now call a most general polymorphic type, consisting of a type expression with universal quantifiers as in Section 6.6. Motivated by Curry, Hindley [1969] observed that unification could be used to infer the types. Independently, in his thesis, Morris [1968a] assigned types to lambda expressions by setting up a set of equations and solving them to determine the types associated with variables. Unaware of Hindley's work, Milner [1978] also observed that unification could be used to solve the sets of equations, and applied the idea to infer types in the ML programming language.

The pragmatics of type checking in ML are described by Cardelli [1984]. This approach has been applied to a language by Meertens [1983]; Suzuki [1981] explores its application to Smalltalk 1976 (Ingalls [1978]). Mitchell

[1984] shows how coercions can be included.

Morris [1968a] observes that recursive or circular types allow types to be inferred for expressions containing the application of a function to itself. The C program in Fig. 6.28, containing an application of a function to itself, is motivated by an Algol program in Ledgard [1971]. Exercise 6.31 is from MacQueen, Plotkin, and Sethi [1984], where a semantic model for recursive polymorphic types is given. Different approaches appear in McCracken [1979] and Cartwright [1985]. Reynolds [1985] surveys the ML type system, theoretical guidelines for avoiding anomalies involving coercions and overloading, and higher order polymorphic functions.

Unification was first studied by Robinson [1965]. The unification algorithm in Section 6.7 can readily be adapted from algorithms for testing equivalence of (1) finite automata and (2) linked lists with cycles (Knuth [1973a], Section 2.3.5, Exercise 11). The almost linear algorithm for testing equivalence of finite automata due to Hopcroft and Karp [1971] can be viewed as an implementation of the sketch on p. 594 of Knuth [1973a]. Through clever use of data structures, linear algorithms for the acyclic case are presented by Paterson and Wegman [1978] and by Martelli and Montanari [1982]. An algorithm for finding congruent nodes (see Exercise 6.30) appears in Downey, Sethi, and Tarjan [1980].

Despeyroux [1984] describes a type-checker generator that uses pattern matching to create a type checker from an operational-semantic specification based on inference rules.

Run-Time Environments

Before considering code generation, we need to relate the static source text of a program to the actions that must occur at run time to implement the program. As execution proceeds, the same name in the source text can denote different data objects in the target machine. This chapter examines the relationship between names and data objects.

The allocation and deallocation of data objects is managed by the *run-time support* package, consisting of routines loaded with the generated target code. The design of the run-time support package is influenced by the semantics of procedures. Support packages for languages like Fortran, Pascal, and Lisp can be constructed using the techniques in this chapter.

Each execution of a procedure is referred to as an *activation* of the procedure. If the procedure is recursive, several of its activations may be alive at the same time. Each call of a procedure in Pascal leads to an activation that may manipulate data objects allocated for its use.

The representation of a data object at run time is determined by its type. Often, elementary data types, such as characters, integers, and reals, can be represented by equivalent data objects in the target machine. However, aggregates, such as arrays, strings, and structures, are usually represented by collections of primitive objects; their layout is discussed in Chapter 8.

7.1 SOURCE LANGUAGE ISSUES

For specificity, suppose that a program is made up of procedures, as in Pascal. This section distinguishes between the source text of a procedure and its activations at run time.

Procedures

A *procedure definition* is a declaration that, in its simplest form, associates an identifier with a statement. The identifier is the *procedure name*, and the statement is the *procedure body*. For example, the Pascal code in Fig. 7.1 contains the definition of the procedure named `readarray` on lines 3-7; the body of the procedure is on lines 5-7. Procedures that return values are called *functions* in many languages; however, it is convenient to refer them as

```
(1) program sort(input, output);
(2)   var a : array [0..10] of integer;
(3)   procedure readarray;
(4)     var i : integer;
(5)     begin
(6)       for i := 1 to 9 do read(a[i])
(7)     end;
(8)   function partition(y, z: integer) : integer;
(9)     var i, j, x, v: integer;
(10)    begin ...
(11)    end;
(12)  procedure quicksort(m, n: integer);
(13)    var i : integer;
(14)    begin
(15)      if ( n > m ) then begin
(16)        i := partition(m,n);
(17)        quicksort(m,i-1);
(18)        quicksort(i+1,n)
(19)      end
(20)    end;
(21)  begin
(22)    a[0] := -9999; a[10] := 9999;
(23)    readarray;
(24)    quicksort(1,9)
(25)  end.
```

Fig. 7.1. A Pascal program for reading and sorting integers.

procedures. A complete program will also be treated as a procedure.

When a procedure name appears within an executable statement, we say that the procedure is *called* at that point. The basic idea is that a procedure call executes the procedure body. The main program in lines 21-25 of Fig. 7.1 calls the procedure `readarray` at line 23 and then calls `quicksort` at line 24. Note that procedure calls can also occur within expressions, as on line 16.

Some of the identifiers appearing in a procedure definition are special, and are called *formal parameters* (or just *formals*) of the procedure. (C calls them "formal arguments" and Fortran calls them "dummy arguments.") The identifiers `m` and `n` on line 12 are formal parameters of `quicksort`. Arguments, known as *actual parameters* (or *actuals*) may be passed to a called procedure; they are substituted for the formals in the body. Methods for setting up the correspondence between actuals and formals are discussed in Section 7.5. Line 18 of Fig. 7.1 is a call of `quicksort` with actual parameters `i+1` and `n`.

Activation Trees

We make the following assumptions about the flow of control among procedures during the execution of a program:

1. Control flows sequentially; that is, the execution of a program consists of a sequence of steps, with control being at some specific point in the program at each step.
2. Each execution of a procedure starts at the beginning of the procedure body and eventually returns control to the point immediately following the place where the procedure was called. This means the flow of control between procedures can be depicted using trees, as we shall soon see.

Each execution of a procedure body is referred to as an activation of the procedure. The *lifetime* of an activation of a procedure p is the sequence of steps between the first and last steps in the execution of the procedure body, including time spent executing procedures called by p , the procedures called by them, and so on. In general, the term "lifetime" refers to a consecutive sequence of steps during the execution of a program.

In languages like Pascal, each time control enters a procedure q from procedure p , it eventually returns to p (in the absence of a fatal error). More precisely, each time control flows from an activation of a procedure p to an activation of a procedure q , it returns to the same activation of p .

If a and b are procedure activations, then their lifetimes are either non-overlapping or are nested. That is, if b is entered before a is left, then control must leave b before it leaves a .

This nested property of activation lifetimes can be illustrated by inserting two print statements in each procedure, one before the first statement of the procedure body and the other after the last. The first statement prints *enter* followed by the name of the procedure and the values of the actual parameters; the last statement prints *leave* followed by the same information. One execution of the program in Fig. 7.1 with these print statements produced the output shown in Fig. 7.2. The lifetime of the activation `quicksort(1,9)` is the sequence of steps executed between printing *enter quicksort(1,9)* and printing *leave quicksort(1,9)*. In Fig. 7.2, it is assumed that the value returned by `partition(1,9)` is 4.

A procedure is *recursive* if a new activation can begin before an earlier activation of the same procedure has ended. Figure 7.2 shows that control enters the activation of `quicksort(1,9)`, from line 24, early in the execution of the program but leaves this activation almost at the end. In the meantime, there are several other activations of `quicksort`, so `quicksort` is recursive.

A recursive procedure p need not call itself directly; p may call another procedure q , which may then call p through some sequence of procedure calls. We can use a tree, called an *activation tree*, to depict the way control enters and leaves activations. In an activation tree,

```

Execution begins...
enter readarray
leave readarray
enter quicksort(1,9)
enter partition(1,9)
leave partition(1,9)
enter quicksort(1,3)
...
leave quicksort(1,3)
enter quicksort(5,9)
...
leave quicksort(5,9)
leave quicksort(1,9)
Execution terminated.

```

Fig. 7.2. Output suggesting activations of procedures in Fig. 7.1.

1. each node represents an activation of a procedure,
2. the root represents the activation of the main program,
3. the node for a is the parent of the node for b if and only if control flows from activation a to b , and
4. the node for a is to the left of the node for b if and only if the lifetime of a occurs before the lifetime of b .

Since each node represents a unique activation, and vice versa, it is convenient to talk of control being at a node when it is in the activation represented by the node.

Example 7.1. The activation tree in Fig. 7.3 is constructed from the output in Fig. 7.2.¹ To save space, only the first letter of each procedure is shown. The root of the activation tree is for the entire program `sort`. During the execution of `sort`, there is an activation of `readarray`, represented by the first child of the root, with label `r`. The next activation, represented by the second child of the root, is for `quicksort`, with actuals 1 and 9. During this activation, the calls of `partition` and `quicksort` on lines 16-18 of Fig. 7.1 lead to the activations `p(1,9)`, `q(1,3)`, and `q(5,9)`. Note that the activations `q(1,3)` and `q(5,9)` are recursive, and that they begin and end before `q(1,9)` ends. □

¹ The actual calls made by `quicksort` depend on what `partition` returns (see Aho, Hopcroft, and Ullman [1983] for details of the algorithm). Figure 7.3 represents one possible tree of calls. It is consistent with Fig. 7.2, although certain calls low in the tree are not shown in Fig. 7.2.